

Gecentraliseerd en Geautomatiseerd Bestandsbeheer: Validatie en Generatie

gemakkelijk info opvragen uit foto's en elektrische
schema's

Door: Robin Herickx

Hogeschoolpromoter: Glenn Adriaensens

Bedrijfspromoter: Veronique sauviller.

PBA Elektronica-ICT
Applied Artificial Intelligence

**Bachelorproef voorgedragen tot het behalen
van de graad en het diploma van bachelor**

Campus De Nayer

Inhoudstafel

INLEIDING	13
PROBLEEMSTELLING	14
1.KNOWLEDGE GRAPH EN NEO4J	15
1.1 Neo4J EN TRADITIONELE RAG-SYSTEMEN	15
1.2 KNOWLEDGE GRAPH METHODES	16
1.3 DE LAY-OUT VAN DE KNOWLEDGE GRAPH	17
2.RAG (RETRIEVAL-AUGMENTED GENERATION)	19
2.1 WAT IS RAG	19
2.1.1 VECTOR SIMILARITY SEARCH	20
2.1.1.1 HIERARCHICAL NAVIGABLE SMALL WORLD (HNSW) ALGORITME	20
2.1.1.2 DISTANCE METRICS	21
2.2 IMPROVING THE RAG PIPELINE	22
2.2.1 RERANKING	23
2.2.2 LLM-BASED DOCUMENT EXPANSION	24
2.2.3 TOOL CALLING BINNEN RAG	26
3.OCR (OPTICAL CHARACTER RECOGNITION)	27
3.1 OCR-OPTIMALISATIES: ENSEMBLE OCR EN TILING	27
4.RRF (RECIPROCAL RANK FUSION)	28
4.1 WEIGHTED RRF	29
6.BAYESIAN OPTIMIZATION	30
6.DATA VERZAMELEN	32
6.1 COMPONENT DATA UIT DWG-BESTANDEN	32
6.2 DE OPLOSSING: WERKEN MET OCR EN SMART PDF's	34
6.3 VERZAMELEN VAN ONDERDEEL-DOCUMENTATIE	35
7.TEST METHODE EN TEST SETS	37
7.1 MAGAZIJN-TESTSET	37
7.1.1 DE ORIGINELE TESTSET	38
7.1.2 SYNTHETISCHE TESTSET	39
7.1.3 MEERDERE SYNTHETISCHE TESTSETS / MODEL EVALUATIE	42
8.MAGAZIJN EN OCR RAG PIPELINE	43
8.1 DOEL	43
8.2 AANPAK	43
9.RESULTATEN	47
9.1 EMBEDDING MODELLEN OP DE SYNTHETISCHE MAGAZIJN TESTSETS	47
9.1.1 EVALUATIE VAN DE EMBEDDING MODELLEN	51
9.1.2 RUIS ROBUSTNESS TEST	51
9.1.3 WEIGHTED RRF	54
9.2 CONCLUSIE EMBEDDING MODELS EN RRF	58
10.OVERZICHT PROJECT EN CONCLUSIE	59
10.1 USER INTERFACE	59
10.1.1 ZOEKEN OP FOTO	59
10.1.2 INFOPAGINA	61

10.1.3 CHATBOT PAGINA	63
10.1.4 ZOEKEN MET TECHNISCHE SCHEMA	65
10.2 ELEKTRISCHE SCHEMA'S, SHAREPOINT EN SMART PDF's	67
10.3 BEHAALDE DOELSTELLINGEN EN AFWIJKING VAN HET OPSLAG VERSLAG	67
10.3.1 PLC CODE	67
10.3.2 ONDERDELEN VINDEN IN HET MAGAZIJN	68
10.4 ONGEBRUIKTE METHODES	68
10.4.1 DWG-EXTRACTIE	68
10.4.2 IMAGE RECOGNITION	69
 CONCLUSIE	 70
LITERATUURLIJST	71

Woord vooraf

Graag wil ik alle leerkrachten bedanken die mij geholpen hebben met mijn studies, in het bijzonder Johan Van Bauwel, Daiane Seibert en Sofie Beerens. Mijn dank gaat ook uit naar mijn Hogeschool Promoter, Glenn Adriaensens, en mijn Bedrijfspromotor, Veronique Sauviller. Ten slotte wil ik het bedrijf Lonza bedanken voor het mogelijk maken van deze stage.

Samenvatting

Het doel van dit project is het centraliseren van bedrijfsgegevens en het eenvoudiger maken om onderdelen terug te vinden in het magazijn. Daarnaast wordt een lokale chatbot gemaakt die gebruikers toelaat om gerichte vragen te stellen over de documentatie van onderdelen. De centrale probleemstelling van dit onderzoek is:

wat is de meest efficiënte manier om bedrijfsdata(technische schema's)

te representeren en onderling te koppelen(documentatie/magazijn info)?

Een belangrijke vereiste binnen dit project is dat alle relevante informatie uit diverse bestandstypes geëxtraheerd kan worden. Dit omvat niet alleen tekst, afbeeldingen en tabellen, maar ook componenten uit elektrische schema's (DWG). Bovendien moet deze informatie op een correcte en consistente manier met elkaar verbonden worden. Zo moeten componenten uit technische tekeningen gekoppeld zijn aan bijbehorende documentatie, die op zichzelf aanvullende informatie bevat en gelinkt is aan andere documenten, zoals gegevens over magazijnlocatie en voorraad.

Om dit te realiseren werden verschillende tools ontwikkeld. Er zijn tools gemaakt voor data-extractie uit technische schema's en documentatie, waaronder een web scraper voor het verzamelen van aanvullende documentatie over onderdelen. Er werd ook een systeem opgezet om alle verzamelde data te structureren en te integreren binnen een knowledge graph

en er is een tool gemaakt voor het genereren van smart pdfs waarbij mensen op een onderdeel in een tekening kunnen klikken en alle info zien

Tenslotte is er ook een tool gemaakt waarbij een foto genomen kan worden van onderdelen en van daaruit de magazijnlocatie en documentatie teruggevonden wordt.

Lijst met afkortingen en symbolen

Afkorting	Betekenis
ANN	Approximate Nearest Neighbor (zoek methode)
DWG	Bestandstype voor technische tekeningen (elektrische schema's)
DXF	Drawing Exchange Format (formaat voor DWG-bestanden)
HNSW	Hierarchical Navigable Small World (Algoritme)
JSON	JavaScript Object Notation (bestandsformaat)
LLM	Large Language Model
LPG	Labeled Property Graph
OCR	Optical Character Recognition
PLC	Programmable Logic Controller
RAG	Retrieval Augmented Generation
RDF	Resource Description Framework
RRF	Reciprocal Rank Fusion
SAP-nummer	Uniek nummer dat aan elk onderdeel in het magazijn is gekoppeld
VRAM	Video Random Access Memory
WRRF	Weighted Reciprocal Rank Fusion

Lijst met figuren

Figuur 1. Visuele structuur van een component in de graph database	18
Figuur 2. Layout basic RAG	19
Figuur 3. DWG-extractie pipeline	32
Figuur 4. JSON tabel waar data niet is ingevuld	33
Figuur 5. JSON tabel waar data wel is ingevuld	33
Figuur 6. Visualisatie van de data extractie van een DWG file	34
Figuur 7. Visualisatie van herkende componenten door middel van OCR	35
Figuur 8. Pipeline om documenten te downloaden via een web scraper	36
Figuur 9. Flowchart Synthetische testset generatie	41
Figuur 10. Bayesian optimization config voor RRF	45
Figuur 11. OCR RRF flowchart	46
Figuur 12. pagina om onderdelen terug te vinden vanuit een foto	60
Figuur 13. debug menu	60
Figuur 14. pagina waar de gebruiker info kan zien van een opgehaald product	62
Figuur 15. magazijn layout waarbij de component locatie is aangeduid	62
Figuur 16. documentatie preview	63
Figuur 17. chatbot interface	64
Figuur 18. instellingenmenu	64
Figuur 19. LLM model selectie	64
Figuur 20. slider om de parameters van het model aan te passen	65
Figuur 21. extra ingeligen en ui optimalisaties	65
Figuur 22. pagina om smart pdf te genereren	66
Figuur 23. pagina om smart pdf te bekijken	66

Opstart Verslag

Gegevens student(en)
Student 1: Robin Herickx. Opleiding: ICT Applied Artificial Intelligence (AI) thomas more de nayer GSM: 0468294560
Gegevens bedrijf / instelling
Naam: Lonza Adres: Rijksweg 11, 2880 Bornem Tel: 038900757
Gegevens bedrijfspromotor
Naam: Veronique sauviller. Tel: 038900757 Mail: Veronique.sauviller@Lonza.com Functie: baas m&e Afdeling: M&E

Gegevens bachelorproef

Titel bachelorproef

[Geautomatiseerde Centralisatie en Correctie/generen van Bestanden](#)

Omschrijving van het probleem

Projectbeschrijving

Lonza beschikt over diverse installaties en machines, waarbij de bijbehorende documentatie verspreid is over verschillende, vaak ongeorganiseerde, mappen. Dit leidt tot problemen zoals:

- Documenten die in de verkeerde map staan.
- Ontbrekende documenten.
- Documenten die nog geconverteerd moeten worden naar het juiste bestandsformaat.

Het bedrijf wil deze situatie verbeteren door een tool te ontwikkelen die de documentenstroom centraliseert en automatiseert via de bestaande SharePoint-omgeving.

En ai gebruiken om de nodige ontbrekende files te generen

Doelstellingen van de Tool

De te ontwikkelen tool heeft de volgende hoofddoelstellingen:

1. **Centralisatie en Correctie:** Alle documenten van een specifieke installatie combineren, corrigeren en automatisch op de SharePoint-site plaatsen.
2. **Volledigheidscontrole:** Controleren of een installatie alle noodzakelijke en vereiste documenten bevat.
3. **Formaatconversie:** Documenten automatisch omzetten naar het juiste type (bijvoorbeeld, zorgen dat elke DWG een bijbehorende PDF-versie heeft).
4. **Generatie van Ontbrekende Documenten:** met ai Ontbrekende documenten automatisch genereren op basis van de bestaande documenten van de betreffende installatie.

Aanpak

Om de doelstellingen te bereiken, zal het project een **lokaal RAG-model** (Retrieval-Augmented Generation) implementeren.

- **Informatie-extractie:** De tool moet informatie uit verschillende bestandstypes kunnen halen, waaronder PDF's, DWG's (technische en elektrische schema's), gescande documenten, afbeeldingen en PLC-code. Er moeten specifieke tools worden ontwikkeld om alle relevante data uit deze bestanden te extraheren en aan het RAG-model te voeden.
- **Generatieve Functionaliteit:** Op basis van de geëxtraheerde informatie kunnen ontbrekende bestanden automatisch worden gegenereerd
- (bv. het genereren van een functionele beschrijving op basis van PLC-code).
- **Interactieve Query's:** De tool moet gebruikers in staat stellen om vragen te stellen aan het RAG-model over de installatiedocumentatie()

Voorbeeld: DWG-file

1. Een zelfgemaakte tool laadt de informatie, componenten en component nummers van een DWG in het RAG-model (het model werkt met een gegenereerde informatie in plaats van de ruwe DWG).
2. Indien er geen PDF-versie van de DWG (elektrisch plan) bestaat, wordt deze aangemaakt.
3. Een medewerker kan de PDF op een tablet openen en vragen stellen aan het RAG-model (bijvoorbeeld: "Wat doet component nummer A12?").
4. Het model zoekt lokaal naar het component en kan indien nodig aanvullende openbare informatie ophalen (via browser mcp), zonder bedrijfsdocumenten online te delen.
5. (beter optie is om zo veel mogelijk documentatie te web scrapen en lokaal te bewaren zo dat mijn model volledig offline werkt)

5. Functionaliteiten en Gebruikerscontrole

De tool moet een visuele interface bieden, waarbij de gebruiker te allen tijde volledige controle behoudt over het beheer en de verwerking van de documenten en data.

De gebruiker moet controle hebben over:

- De plaatsing van bestanden (bij welke installatie).
- Het samenvoegen van mappen of bestanden.
- Het overzicht van ontbrekende, aan te passen of te converteren documenten.
- Welke documenten potentieel gegenereerd kunnen worden (bv. functionele beschrijvingen uit PLC-code en dwg).

Doelstellingen van de bachelorproef (kwantitatief en meetbaar)

1. RAG-model bestandsondersteuning: Het RAG-model moet alle relevante bestandstypes kunnen uitlezen en indien nodig metadata-bestanden genereren. (dwg, pdf, odt, png, txt)
2. Uittesten/maken van verschillende RAG-pipelines en LLM/NLP-methodes (geven complexere pipelines betere resultaten en wat past het best bij ons?) Testen van verschillende lokale LLM's en embedding-modellen (een custom lokaal LLM trainen/fine-tunen op bedrijfsdata)
3. Tools/tests bouwen om methodes te vergelijken en te documenteren
bv. methode A geeft betere, bedrijfsgerichte antwoorden dan methode B, maar is 10x trager
Methodes/modellen documenteren en vergelijken op basis van:
 - 3.1. Correctheid (op basis van een testset)
 - 3.2. Complexiteit en uitbreidbaarheid
 - 3.3. Responstijd
 - 3.4. Compute needed to run
 - 3.5. ...
4. UI bouwen voor de gebruiker

Gewenst resultaat: wat moet er (minimaal) opgeleverd worden?

1. Een werkende tool die documenten van verschillende installaties kan samenvoegen, corrigeren/generen en op SharePoint kan zetten.
2. De tool kan documenten automatisch converteren (bijv. DWG → PDF) en andere bestandstypes correct verwerken, inclusief het genereren van extra metadata- bestanden die het RAG-model kan lezen.
3. Ontbrekende bestanden kunnen automatisch worden gegenereerd bv.functionele beschrijving
4. Het RAG-model draait lokaal en kan vragen beantwoorden over componenten en PLC-code, zonder bedrijfsdocumenten online te delen.
5. De tool toont visueel welke documenten ontbreken, aangepast moeten worden of geconverteerd moeten worden.
6. Minimaal één testinstallatie is volledig verwerkt en gecontroleerd met de tool.
7. een rapport met alle methodes/models die ik geprobeerd heb en wat mijn eind resultaat was en welke methode die ik uiteindelijk heb toe gepast

Wat is de planning (activiteiten van de student + includeer ook planning m.b.t. het schrijven van de scriptie)?

Algemene Planning

Fase 1: ongeveer de eerste 4-8 weken: het opzetten van een lokaal RAG-model. Onderzoek doen naar welke opties er allemaal zijn, een tool maken voor het uitlezen van DWG-bestanden, een tool maken voor het scrapen van online documenten, en onderzoek doen/tools maken om de prestatie van mijn model te meten/bij te houden. Voor de scriptie ben ik van plan deze elke week tot twee weken te updaten met de tools die ik gemaakt heb en de documentatie daarover.

Fase 2: ongeveer 3-5 weken: verschillende soorten RAG testen en verschillende lokale LLM's testen, en de prestatie (methodes/modellen) documenteren in mijn scriptie.

Fase 3: alles afwerken en het afronden van mijn scriptie.
(Het kan zijn dat Fase 1 en 2 uitlopen of korter zijn; moeilijk in te schatten in het begin.)

Handtekening Student: 	Handtekening bedrijfs promotor: 	Handtekening hogeschoolpromotor Glenn Adriaenssens Digitally signed by Glenn Adriaenssens Date: 2026.02.19 10:21:24 +01'00'
--	--	---

Inleiding

In deze scriptie wordt een antwoord gezocht op de volgende onderzoeksvraag: **wat is de meest efficiënte manier om bedrijfsdata, zoals technische schema's, te representeren en onderling te koppelen?**

Het doel is om informatie sneller en eenvoudiger toegankelijk te maken binnen een bedrijfsomgeving. Hiervoor wordt een systeem ontwikkeld waarbij technische schema's interactief gebruikt kunnen worden. Via een smart PDF kan bijvoorbeeld op een onderdeel geklikt worden, waarna automatisch wordt doorgestuurd naar een informatiepagina met de locatie van het component in het magazijn en de bijbehorende documentatie.

Daarnaast wordt onderzocht hoe een chatbot gebruikt kan worden om vragen te beantwoorden over documentatie. Het systeem laat ook toe om een foto van een onderdeel te nemen, waarna automatisch naar dezelfde informatiepagina wordt verwezen. Hierdoor wordt het zoeken naar onderdelen aanzienlijk eenvoudiger.

Om dit mogelijk te maken, wordt een knowledge graph ontwikkeld die alle informatie van het bedrijf met elkaar verbindt. Hiervoor wordt gebruikgemaakt van Neo4j.

In deze scriptie wordt onderzocht hoe technische tekeningen verwerkt en toegevoegd kunnen worden aan de knowledge graph. Daarnaast wordt bekeken wat de meest efficiënte manier is om data binnen de knowledge graph op te zoeken met behulp van OCR .

Er worden twee verschillende vector search-methoden vergeleken. Een standaard vector search met een master embedding. een methode op basis van Reciprocal Rank Fusion (RRF), waarbij meerdere zoekresultaten worden gecombineerd om tot een beter eindresultaat te komen.

Probleemstelling

Het opzoeken van informatie over componenten kost in de praktijk vaak veel tijd. Een voorbeeld hiervan is de situatie waarin een bepaald onderdeel van een machine vervangen moet worden. Eerst moet het elektrisch schema van de installatie worden opgezocht. Vervolgens moet worden nagegaan welk component vervangen moet worden, waar het zich in het magazijn bevindt en of het op voorraad is. Indien bijkomende informatie nodig is over het onderdeel, moet ook de bijbehorende documentatie worden opgezocht en doorzocht om de vereiste informatie te vinden.

Wanneer het component niet op voorraad is en tijdelijk een alternatief onderdeel nodig is, moet opnieuw worden nagegaan welke alternatieven beschikbaar zijn. Vervolgens moet de documentatie van deze alternatieve componenten worden geraadpleegd om te beoordelen of ze een correcte vervanging kunnen vormen. In de praktijk moet dit proces vaak voor elk afzonderlijk component opnieuw worden uitgevoerd.

Onderzoek

1. Knowledge graph en Neo4j

Dit hoofdstuk licht kort toe wat een knowledge graph is, waarom deze datastructuur geschikt is voor dit project en hoe de graph is opgebouwd in Neo4j.

1.1 Neo4j en traditionele RAG-systemen

In een traditioneel RAG-systeem wordt meestal gebruikgemaakt van een traditionele vector database. Datapunten worden opgeslagen als vector embeddings. Op basis van een query wordt vervolgens via similarity search het meest relevante datapunt opgehaald.

Deze methode is geschikt voor het terugvinden van semantisch gelijkaardige tekstfragmenten, maar biedt slechts beperkte ondersteuning voor expliciete relaties tussen datapunten. Een tekstfragment kan wel worden teruggevonden maar de koppeling met gerelateerde documenten, onderdelen of locaties vereist bijkomende complexe logica.

Daarom is er gekozen voor Neo4j. Neo4j is een graph-database waarin data wordt opgeslagen als nodes en relaties. Dit maakt het mogelijk om niet alleen afzonderlijke datapunten op te slaan, maar ook de relaties/verbindingen ertussen.

een node kan meerdere eigenschappen bevatten, waaronder tekstuele informatie en vector embeddings. een tekst node heeft de volgende attributen:

- de originele tekst;
- een vector embedding van de tekst;
- een tekstuele beschrijving;
- een vector embedding van de beschrijving.

De kracht van Neo4j ligt in de combinatie van vector search met Cypher-query's. Hierdoor kan eerst een semantische zoekactie worden uitgevoerd op basis van een embedding, waarna de bijbehorende node wordt opgehaald en bijkomende informatie kan worden geraadpleegd. Tegelijk kan de zoekopdracht worden beperkt tot specifieke node types of tot nodes die gekoppeld zijn aan een bepaald document.

Deze aanpak maakt het mogelijk om efficiënt gerelateerde nodes op te vragen. Daardoor kunnen verbanden tussen verschillende entiteiten rechtstreeks worden benut binnen het zoekproces.

1.2 knowledge graph methodes

Er zijn verschillende soorten knowledge graphs en verschillende manieren om deze op te zetten. Hier zijn de drie opties die werden onderzocht:

1. Semantic Web Approach:

Resource Description Framework (RDF) Dit is de traditionele, academische methode om een knowledge graph op te bouwen. Data wordt opgeslagen in zogenaamde "triples", die bestaan uit een onderwerp, een predicaat en een object (bijvoorbeeld: Document A -> behoort tot -> Categorie B). Deze aanpak maakt gebruik van strikte vocabulaires en ontologieën om relaties vast te leggen [\[ref1\]](#). Het grote voordeel van RDF is de standaardisatie, wat het makkelijk maakt om verschillende datasets met elkaar te combineren. Het nadeel is echter dat het systeem complex is om te leren en dat het doorzoeken van diepe relaties in grote datasets traag kan zijn.

2. Labeled Property Graph (LPG) Approach:

Dit is de standaard binnen de industrie voor moderne applicaties en is ook het model dat Neo4j gebruikt. In een LPG wordt de data gestructureerd als nodes (entiteiten) en edges (relaties). In tegenstelling tot RDF kunnen in een LPG zowel de nodes als de edges meerdere properties bevatten. Dit zorgt ervoor dat data heel compact kan worden zonder dat er onnodig extra nodes hoeven te worden aangemaakt. Het is een zeer intuïtieve manier van modelleren en biedt extreem snelle prestaties bij het doorzoeken van connecties in de graph.

3. Knowledge Hypergraphs

Waar een standaard graph (zoals RDF of LPG) gebruik maakt van binaire relaties waarbij een edge altijd exact twee nodes verbindt, generaliseert een hypergraph dit concept. In een hypergraph kan één enkele relatie (een hyper edge) meerdere nodes tegelijkertijd met elkaar verbinden [\[ref 2\]](#). Dit is nuttig voor complexe situaties. Het nadeel is dat er veel minder tools voor beschikbaar zijn in vergelijking met LPG's.

note: over automatische generatie met LLM-tools: Neo4j biedt een bestaande tool aan (de LLM Knowledge Graph Builder) voor het automatisch genereren van knowledge graphs vanuit tekst. De reden waarom ervoor is gekozen om deze tool niet te gebruiken, is omdat deze afhankelijk is van Large Language Models (LLM's) voor het genereren van de nodes en relaties. In de praktijk zorgt dit ervoor dat de database heel snel onoverzichtelijk en chaotisch wordt, zeker wanneer er met grotere datasets wordt gewerkt [\[ref 3\]](#). De LLM creëert vaak inconsistente benamingen en dubbele relaties, waardoor de structuur van de graph verloren gaat.

1.3 De lay-out van de knowledge graph

Elk datapunt binnen de knowledge graph (KN) bestaat uit nodes. Elke node bevat verschillende properties. In deze sectie wordt de uiteindelijke structuur van de KN toegelicht, inclusief de aanwezige nodes, de properties van elke node en de relaties tussen de nodes.

De KN bestaat voornamelijk uit component nodes. Deze nodes stellen alle componenten en onderdelen voor die door het bedrijf worden aangeleverd. Elke component node bevat verschillende properties, zoals [Manufacturer_Part_No, Material_Description, SAPnumber].

Voor het genereren van deze component nodes wordt gebruikgemaakt van een XLS-bestand dat door het bedrijf is aangeleverd. Dit bestand bevat informatie over het magazijn en de onderdelen.

Bijna elke component node is gekoppeld aan een PDF document node (zie sectie 6.3). Een PDF node is opgesplitst in pagina nodes, waarbij elke pagina node verder wordt onderverdeeld in tabel, tekst en afbeelding nodes. Voor het extraheren van tekst chunks, tabellen en afbeeldingen uit de PDF's is gebruikgemaakt van Dockling [\[ref 18\]](#).

Daarnaast zijn er smart PDF nodes die alle informatie van een technische tekening bevatten (zie sectie 6.2). Deze nodes zijn gekoppeld aan de component nodes.

Fig 1 is een visuele structuur van hoe een component in de graph database wordt opgeslagen.

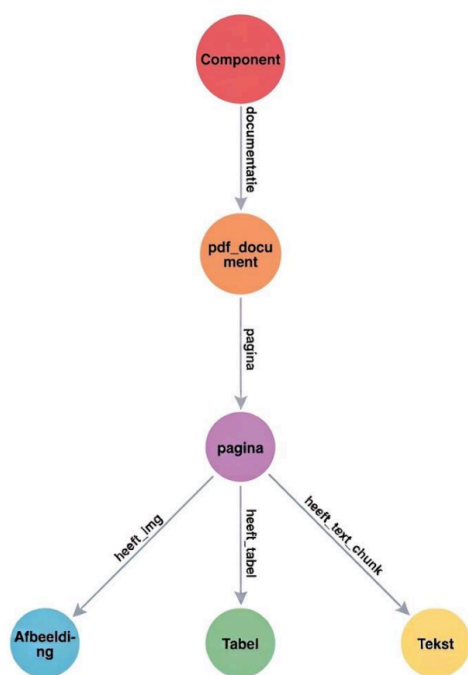


fig 1 Visuele structuur van een component in de graph database

2. RAG(Retrieval-Augmented Generation)

Hier in dit hoofdstuk wordt besproken wat RAG is, de verschillende varianten, wat de voor- en nadelen zijn van elke methode, de uiteindelijke RAG-pipeline

2.1 Wat is RAG?

RAG [\[ref 8\]](#) is een methode om een LLM van extra context te voorzien.

Een eenvoudige RAG-pipeline werkt als volgt. Eerst is er een user query. In plaats van deze rechtstreeks aan een LLM te geven, wordt de query eerst doorgegeven aan een retrieval model. Dit model zet de query om in een embedding. Met deze embedding wordt vervolgens een top-k vector similarity search uitgevoerd op de embeddings van de tekst chunks [\[2.1.1\]](#).

Daarna worden de tekstfragmenten van de top-k resultaten geselecteerd en toegevoegd aan de oorspronkelijke query. Deze uitgebreide query wordt vervolgens doorgegeven aan de LLM, zodat het model een antwoord kan genereren op basis van zowel de vraag als de opgehaalde context.

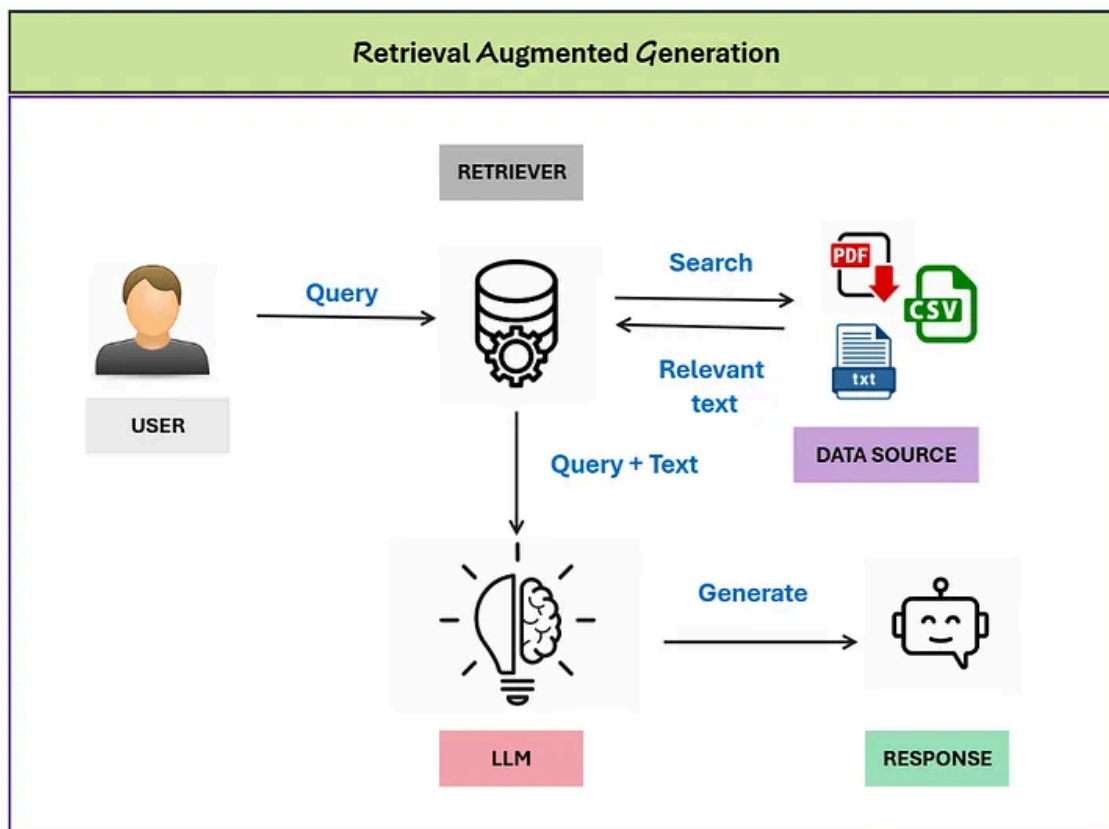


fig 2 layout basic rag [\[afbeelding source\]](#)

2.1.1 vector similarity search

Voor de vector similarity search werd gebruikgemaakt van de Neo4j similarity function [\[ref4\]](#). Dit is een ingebouwde functie van Neo4j waarmee nodes een vector property kunnen bevatten. Op basis van deze vector properties kan vervolgens een vector index worden aangemaakt voor alle embeddings.

Binnen Neo4j werkt dit als volgt: nadat aan alle relevante nodes een vector property is toegevoegd, wordt een vector index opgebouwd over deze vectors. Neo4j gebruikt hiervoor het HNSW-algoritme [4.1.1.1] in combinatie met een distance metric [4.1.1.2] om de top-k nodes te vinden waarvan de vector het dichtst bij de query vector ligt. Neo4j ondersteunt hierbij zowel de cosine als de euclidean distance metric.

Daarnaast biedt Neo4j ondersteuning voor vector quantization. Hierbij worden vectors gecomprimeerd om het geheugenverbruik te verlagen en de zoekoperatie sneller en efficiënter te maken. Het gevolg hiervan is dat de zoekresultaten minder nauwkeurig kunnen worden.

2.1.1.1 Hierarchical Navigable Small World (HNSW) Algoritme

Het HNSW algoritme is een datastructuur die wordt gebruikt voor Approximate Nearest Neighbor (ANN) search in high dimensional vector spaces. Neo4j gebruikt HNSW als basis voor vector indexing. Een vector index wordt aangemaakt met `create vector index` deze heeft de parameters `vector.hnsw.m` en `vector.hnsw.ef_construction` `vector.hnsw.m` bepaalt het maximum aantal verbindingen per node in de HNSW-graph, terwijl `vector.hnsw.ef_construction` aangeeft hoeveel burens tijdens het opbouwen van de index worden bijgehouden. Een hogere waarde voor `vector.hnsw.m` en `vector.hnsw.ef_construction` kan de kwaliteit verbeteren, maar verhoogt ook de opbouw en updatekost.

Hoe werkt HNSW?

In plaats van elke vector met alle andere vectoren te vergelijken, bouwt het algoritme een graph structuur op met meerdere niveaus. De bovenste lagen bevatten minder knopen en dienen om snel grote sprongen te maken, terwijl de onderste laag verfijnd wordt door dichter bij de juiste burens te zoeken. Daardoor kan het algoritme efficiënt zoeken in high dimensional vector spaces. HNSW is volledig graf gebaseerd en werd ontworpen als een snelle en robuuste aanpak voor k-nearest-neighbor search. [\[ref 9\]](#)[\[ref4\]](#)

Bij het zoeken start HNSW bij een entry point in de bovenste laag. Van daaruit beweegt het algoritme greedily naar burens die dichter bij de query liggen. Op elk lager niveau wordt deze zoekstap verder verfijnd, tot het algoritme in de onderste laag de uiteindelijke kandidaten teruggeeft. Tijdens het opbouwen van de index worden nieuwe vectoren laag per laag ingevoegd, waarbij het aantal onderzochte kandidaten tijdens de constructie wordt bepaald door de parameter `efConstruction`. Dit maakt het mogelijk om een goede balans te kiezen tussen index kwaliteit en opbouwtijd.

Voordelen van HNSW

De integratie van HNSW in een vector database biedt substantiële voordelen ten opzichte van traditionele search methodes:

- **Logaritmische Scalability:** De zoektijd complexiteit van HNSW schaalt gemiddeld als $O(\log N)$, waarbij N de dataset size is. Dit maakt milliseconde latency mogelijk bij het doorzoeken van miljoenen vectoren.
- **Hoge Recall Efficiency:** Vergeleken met andere ANN-algoritmes, zoals Inverted File (IVF) indexes of Product Quantization (PQ), levert HNSW een superieure trade-off tussen query throughput en accuracy.
- **Geen Exhaustive Search:** In plaats van een lineaire scan waarbij elke vector wordt vergeleken ($O(N)$), minimaliseert HNSW de compute load door uitsluitend relevante paden in de graph te traverseren.
- **In-index Filtering:** Specifiek binnen Neo4j faciliteert de HNSW-structuur de integratie van metadata filters (pre-filtering) direct tijdens de graph traversal. Dit is cruciaal voor hybride queries, waarbij zowel op semantische vector similarity als op expliciete graph properties wordt gefilterd.

2.1.1.2 Distance metrics

Neo4j ondersteunt 2 verschillende distance metrics voor vector similarity search, waaronder cosine similarity en Euclidean distance.

Cosine similarity meet de hoek tussen twee vectoren en negeert daarbij de grootte (magnitude) van de vectoren. De waarde ligt tussen -1 en 1, waarbij 1 betekent dat de vectoren identiek georiënteerd zijn. Deze metric wordt gebruikt bij tekst embeddings, omdat hier vooral de richting (betekenis) van belang is en niet de absolute grootte van de vector.

Euclidean distance meet de rechte afstand tussen twee vectoren in de vectorruimte. Hoe kleiner de afstand, hoe gelijkwaardiger de vectoren zijn. In tegenstelling tot cosine similarity houdt deze metric wel rekening met de grootte van de vectoren.

Het belangrijkste verschil is dus dat cosine similarity enkel kijkt naar de richting van vectoren, terwijl Euclidean distance zowel richting als grootte in rekening brengt. Hierdoor kunnen twee vectoren met dezelfde richting maar verschillende lengtes als identiek worden beschouwd bij cosine similarity, maar niet bij Euclidean distance.

2.2 mogelijke verbeteringen voor de rag pipeline

Een van de grootste problemen bij standaard RAG is het retrieval gedeelte. Bij een perfecte RAG pipeline worden alle nodige text chunks opgehaald uit de database en bevatten deze alle informatie die nodig is om de vraag correct te beantwoorden. Bij een standaard RAG pipeline is dit meestal niet het geval.

Bij gewone vectorvergelijking, waarbij de prompt als query vector wordt gebruikt, worden text chunks opgehaald die semantisch gelijkwaardig zijn aan de query. Dit betekent echter niet dat deze ook relevant zijn om de vraag te beantwoorden. Een text chunk kan semantisch dicht bij de query liggen, zonder effectief nuttig te zijn.

Als de retriever te weinig of incorrecte context aan de LLM geeft, kan de LLM geen correct antwoord genereren. Om dit te verbeteren, kunnen verschillende technieken worden toegepast.

Een eerste aanpak is om meer datapunten op te halen uit de database, bijvoorbeeld 100 in plaats van 10, en al deze tekst chunks toe te voegen aan de LLM. Het nadeel hiervan is dat de query veel groter wordt. Dit zorgt niet alleen voor langere verwerkingstijden, maar ook voor ruis door irrelevante text chunks.

In plaats van alle text chunks rechtstreeks toe te voegen aan de query, kan een reranking model [2.2.1] gebruikt worden. Dit model neemt de originele query en een grote lijst van opgehaalde text chunks, en rangschikt deze op basis van relevantie voor de query. Na het her-ranken worden enkel de top-k text chunks geselecteerd en toegevoegd aan de originele query.

Een andere verbetering is het aanpassen van de originele query met behulp van een kleine, snelle LLM [2.2.2]. Deze corrigeert grammaticale en taalfouten, wat de kwaliteit van de vector search in de database kan verbeteren.

Daarnaast kan elke text chunk worden verrijkt met een door een LLM gegenereerde beschrijving van de inhoud. Deze beschrijving kan vervolgens ook ge-embedd worden en gebruikt worden voor vector search. Deze techniek heet LLM-based Document Expansion [2.2.2].

Het is belangrijk op te merken dat deze methoden geen perfecte individuele oplossingen zijn. Ze kunnen gecombineerd worden in een grotere en meer complexe RAG pipeline.

2.2.1 Reranking

Een reranker is een machine learning model, gebaseerd op de

Transformer-architectuur, dat getraind is om de relevantie van een document ten opzichte van een query te bepalen. In tegenstelling tot vector search, waarbij query en documenten apart worden omgezet naar embeddings, verwerkt een reranker deze samen als één input. Het model geeft een relevantiescore (tussen 0 en 1) waarmee de resultaten opnieuw gerangschikt worden.

Waarom reranking nodig is in RAG

Reranking lost twee belangrijke problemen op:

- **Semantic mismatch:**
Vector search werkt op basis van afstand tussen embeddings. Dit is goed voor algemene semantische gelijkenis, maar minder geschikt voor complexe context, of specifieke details. Een text chunk kan semantisch dicht bij de query liggen zonder de vraag effectief te beantwoorden
- **Context window en “lost in the middle”:**
LLM's hebben een beperkt context window en kunnen informatie in het midden van een lange prompt missen. Door enkel de meest relevante top-k chunks te selecteren na reranking, wordt ruis beperkt en blijft de belangrijkste informatie behouden.

Technische werking: bi-encoder vs cross-encoder

Om reranking te begrijpen, is het verschil tussen de retriever en reranker belangrijk:

- **Bi-encoders (retrieval):**
De query en documenten worden apart gecodeerd naar vectoren
- De relevantie wordt berekend via cosine similarity [2.1.1.2]. Omdat er geen directe interactie is tussen query en document tijdens encoding, gaat context verloren. Het voordeel is dat deze methode zeer snel is.
- **Cross-encoders (reranking):**
Cross-encoders voor her-rangschikking werken door de zoekopdracht en het document samen in één model invoer te plaatsen. Bijvoorbeeld: eerst een **Classification token** (een speciaal starttoken), daarna de zoekopdracht, vervolgens een **Separator token** (scheidingsstroken), en daarna het document gevolgd door een tweede Separator token. Omdat beide teksten tegelijk door het model worden verwerkt, kan het model volledige aandacht toepassen over alle woorden in beide teksten. Hierdoor kan het niet alleen de algemene betekenis vergelijken, maar ook exacte woord overeenkomsten en subtiele verbanden tussen zoekopdracht en document beter herkennen dan bij enkel semantische gelijkenis.

Het reranking proces

Binnen een RAG-pipeline verloopt dit proces in vier stappen:

1. **Kandidaat-generatie:**
De retriever haalt een grotere set op (bijv. $n = 100$) uit de vector database.
2. **Scoring:**
De reranker (cross-encoder) berekent voor elk (query, chunk)-paar een relevantiescore.
3. **reranking :**
De chunks worden gesorteerd op basis van deze scores.
4. **Selectie:**
Enkel de top-k meest relevante chunks worden doorgestuurd naar de LLM voor generatie

2.2.2 LLM-based Document Expansion

LLM-based Document Expansion is een pre-processing techniek waarbij elke text chunk in de database wordt verrijkt met extra, door een LLM gegenereerde content vóór de embedding wordt aangemaakt. In plaats van enkel de originele tekst te embedden, wordt een uitgebreidere versie gebruikt die extra context, samenvattingen of mogelijke vragen bevat die door de chunk beantwoord kunnen worden [\[ref 10\]](#).

Waarom document expansion nodig is in RAG

Een belangrijk probleem bij standaard RAG is het verschil tussen de vraag van de gebruiker en de opgeslagen tekst:

- **Terminologie-mismatch:**
Een gebruiker stelt een vraag in natuurlijke taal ("Wat is de spanning van dit onderdeel?"), terwijl de database meer formele termen bevat (zoals "nominale spanning", "elektrische specificaties" of "component ratings"). Zonder expansie is de overeenkomst soms te laag.
- **Gebrek aan context:**
Een losse text chunk bevat onvoldoende context (bijvoorbeeld waar "het project" naar verwijst). Document expansion voegt deze context toe.
- **Zoekintentie:**
Door te voorspellen welke vragen een chunk kan beantwoorden, wordt de kans groter dat de retriever de juiste informatie ophaalt bij een gelijkaardige vraag .

Technische werking en methodes

De implementatie van document expansion gebeurt meestal op een van de volgende

- **Doc2Query (query prediction):**
Een LLM genereert meerdere mogelijke vragen voor elke chunk. Deze worden toegevoegd aan de originele tekst. Hierdoor matcht de query van de gebruiker beter met de gegenereerde vragen dan met de ruwe tekst.

Procesverloop

Dit proces gebeurt tijdens de ingestion phase:

1. **Chunking:**
Het document wordt opgesplitst in segmenten.
2. **Enrichment prompting:**
Voor elk segment wordt een prompt naar een LLM gestuurd, bijvoorbeeld:
"Genereer 3 vragen die deze tekst beantwoordt" of
"Vat dit fragment samen in de context van de documenttitel".
3. **Concatenatie:**
De gegenereerde tekst wordt samengevoegd met de originele chunk.
4. **Indexing:**
De uitgebreide chunk wordt ge-embed en opgeslagen in de vector database [\[ref 10\]](#).

2.2.3 Tool Calling binnen RAG

De LLM heeft toegang tot twee tools:

- KN search
- PDF page reader

KN search

Wanneer een vraag onvoldoende informatie bevat, kan de LLM zelfstandig een nieuwe vraag formuleren. Op basis van deze nieuwe vraag wordt opnieuw een vector search uitgevoerd binnen de KN. Hierdoor kan het model iteratief extra informatie ophalen en gericht zoeken naar relevante context. Het model bepaalt hierbij zelf wanneer bijkomende informatie nodig is en welke tool gebruikt moet worden [\[ref 12\]](#)[\[ref 13\]](#).

PDF page reader

De tweede tool is de PDF page reader. Deze tool wordt enkel gebruikt bij grotere multimodale modellen met vision-capabilities.

Alle PDF-documenten worden vooraf verwerkt met [Docking](#) [\[ref 18\]](#). Tijdens deze verwerking probeert Docking de originele structuur van het document zoveel mogelijk te behouden, inclusief tabellen en gerelateerde inhoud. In de meeste gevallen werkt dit correct maar bij complexe tabellen raare layout kan de extractie falen.

Wanneer een tabel niet correct geëxtraheerd wordt kan de inhoud van een tabel opgesplitst worden in meerdere losse text chunks in plaats van één gestructureerde tabel node. Hierdoor verliest het model de relatie tussen de verschillende waarden.

Bijvoorbeeld: een tabel van een motor met belasting en snelheid kan opgesplitst worden in afzonderlijke stukken tekst. Hierdoor is het niet meer duidelijk welke snelheid bij welke belasting hoort. Een vraag zoals:

“Wat is de maximale snelheid van de motor bij een belasting van 10 Nm?”

kan in dat geval niet correct beantwoord worden via de standaard vector search, omdat de onderlinge relatie tussen de waarden verloren gegaan is.

Om dit probleem gedeeltelijk op te lossen, kan de PDF page reader de originele PDF-pagina rechtstreeks aan een vision-model doorgeven. Hierdoor krijgt het model toegang tot de volledige visuele structuur van die pagina.

3. OCR (Optical Character Recognition)

Optical Character Recognition (OCR) is een manier om afbeeldingen van getypte,

hand geschreven of gedrukte tekst om te zetten naar machineleesbare en doorzoekbare tekst. Hierdoor kunnen systemen ongestructureerde visuele data, zoals gescande documenten of foto's, omzetten naar gestructureerde digitale informatie.

Moderne OCR-systemen op basis van deep learning bestaan uit twee gekoppelde processen: tekst detectie en tekstherkenning. In de detectiefase zoekt het model naar regio's in de afbeelding die tekst bevatten en tekent het bounding boxes rond deze gebieden. In de herkenningfase worden de visuele kenmerken binnen elke bounding box geëxtraheerd en via neurale netwerken, een combinatie van Convolutional Neural Networks(CNN) en Recurrent Neural Networks, omgezet naar een sequentiële reeks digitale karakters.

Bij het selecteren van een OCR-engine zijn verschillende opensource-opties beschikbaar, zoals Tesseract(google), Easy OCR en Paddle OCR. Voor dit onderzoek is gekozen voor Paddle OCR. Deze architectuur werd geselecteerd vanwege de goede balans tussen hoge inferentie snelheid en hoge accuraatheid. Paddle OCR maakt gebruik van geavanceerde en lichtgewicht modellen zoals DBNet (Differentiable Binarization) voor nauwkeurige tekstdetectie en CRNN (Convolutional Recurrent Neural Network) voor tekstherkenning.

De effectiviteit van OCR is sterk afhankelijk van de kwaliteit van de invoer afbeelding.

3.1 OCR-optimalisaties:

Ensemble OCR en Tiling

Bij Ensemble OCR worden de output resultaten van meerdere OCR-modellen gecombineerd. Door te stemmen op meerdere resultaten , kunnen individuele fouten van één model worden gecorrigeerd door de zekerheid van andere resultaten. Dit leidt tot een robuustere en nauwkeurigere tekst extractie [[ref 14](#)].

Daarnaast werd de techniek tiling getest om de resolutie te verbeteren zonder de originele afbeelding te downscalen. Bij tiling wordt een grote afbeelding in een raster opgedeeld in kleinere, overlappende segmenten (tiles), die afzonderlijk naar het OCR-model worden gestuurd [[ref 15](#)]. Na het testen werd besloten om deze methode niet te implementeren. Het opdelen van de afbeelding leidde ertoe dat het OCR-model te veel ging hallucineren. Bovendien veroorzaakte tiling aanzienlijke fouten in de post-processing: het betrouwbaar samenvoegen van bounding boxes die door de grenslijnen van de tiles werden gesneden, bleek te foutgevoelig, wat resulteerde in afgebroken woorden en corrupte tekst data.

4. RRF (Reciprocal Rank Fusion)

RRF [\[ref 16\]](#)

Reciprocal Rank Fusion (RRF) is een rank-aggregatie techniek die wordt gebruikt om resultaten van meerdere, onafhankelijke informatiebronnen te combineren (de nodes gevonden op elke property). De uiteindelijke rangschikking wordt uitsluitend gebaseerd op de positie (de “rank”) van een document binnen de resultatenlijsten.

Hoe werkt het?

RRF berekent een uiteindelijke fusie score voor elk item door de reciproke waarde (het omgekeerde) van de behaalde rang posities op te tellen. De formule is als volgt:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

Waarbij:

- d het te beoordelen item is (gevonden neonode).
- R de verzameling van alle geretourneerde resultatenlijsten representeert.
- $r(d)$ de positie/rank van een node d in een ranker r
- (de positie van de node in een van de lijsten r).
- k een optimalisatie constante is. Deze constante voorkomt dat een document met één toppositie een onevenredig grote invloed heeft ten opzichte van documenten die consistent hoog scoren in meerdere lijsten.

Motivatie voor de keuze van RRF

Voor elke bounding box wordt een afzonderlijke vector similarity search per property uitgevoerd. Hierdoor ontstaat de noodzaak om meerdere onafhankelijke resultatenlijsten te combineren. Er is bewust gekozen om niet enkel de node met de hoogste similarity score als definitief resultaat te selecteren, om twee redenen.

Ten eerste kan een model een hoge confidence score toekennen aan een foutieve node (false positive). Wanneer uitsluitend op deze score wordt vertrouwd, kan een incorrect resultaat worden geselecteerd.

Ten tweede ontstaat ambiguïteit wanneer meerdere top-nodes vergelijkbare similarity scores behalen. In deze gevallen biedt de ruwe score onvoldoende onderscheid.

Door RRF toe te passen, wordt dit verminderd. De methode houdt geen rekening met absolute scores, maar met de relatieve positie van nodes in de resultatenlijsten. Nodes die consistent hoog scoren over meerdere properties worden hierdoor bevoordeeld.

Een eenmalige foutieve uitschieter met een hoge score wordt geneutraliseerd door de bredere consensus over verschillende eigenschappen. Om de nauwkeurigheid verder te verhogen, wordt aanvullend Weighted RRF toegepast. Aangezien niet elke property even bepalend is, maakt deze uitbreiding het mogelijk om eigenschappen met een hogere zekerheid zwaarder te laten doorwegen in de finale aggregatie.

4.1 Weighted RRF

Weighted Reciprocal Rank Fusion (WRRF) is een uitbreiding van de standaard RRF-methode. Waar het oorspronkelijke algoritme alle zoekmethoden als gelijkwaardig beschouwt, introduceert WRRF een gewichtsfactor. Hierdoor kan extra nadruk worden gelegd op specifieke rankers(properties), afhankelijk van hun betrouwbaarheid of relevantie binnen het domein.

De wiskundige formulering wordt uitgebreid met een wegingsvariabele:

$$Weighted_RRF(d) = \sum_{r \in R} \frac{w_r}{k + r(d)}$$

Waarbij w_r het gewicht voor ranker r voorstellen(property).

Voordelen

Precisie:

Niet elke property is even belangrijk voor de identificatie van een component. Een exacte match op een eigenschap zoals "Manufacturer Part No" biedt meer zekerheid dan een algemene overeenkomst op "Material Description". Door de gewichtsfactor w_r aan te passen, wordt de invloed van sterk voorspellende eigenschappen vergroot, terwijl minder relevante resultaten worden afgezwakt.

Ruisreductie:

Standaard RRF kan gevoelig zijn voor ruis wanneer een vector similarity search op een bepaalde property zwakke of irrelevante matches oplevert. Weighted RRF beperkt deze invloed door dergelijke resultatenlijsten een lager gewicht toe te kennen. Dit resulteert in een betrouwbaardere en nauwkeurigere selectie van nodes voor de uiteindelijke identificatie

5. Bayesian optimization

Wat is Bayesian optimization? [\[ref 17\]](#)

Bayesian optimization is een probabilistische sequentiële strategie voor het globaal optimaliseren van functies. Binnen de context van het Weighted RRF-proces ontstaat de uitdaging dat bepaalde aannames zoals het veronderstelde hogere belang en de betrouwbaarheid van de properties “Manufacturer Part No” en “Product Name” niet op voorhand bevestigd kunnen worden. In plaats van te steunen op een handmatige of arbitraire selectie van deze wegingsfactoren wordt Bayesian optimization toegepast om de optimale waarden van de hyperparameters op een objectieve en datagedreven manier te bepalen.

Hoe werkt het?

Het model is opgebouwd uit twee componenten:

Surrogaatmodel (Gaussian Process):

Het rechtstreeks evalueren van de volledige tekst extractie- en aggregatie pipeline voor elke mogelijke combinatie van gewichten is computationeel zeer zwaar. Daarom bouwt Bayesian optimization een probabilistisch surrogaat model van de objectieve functie. Een Gaussian Process (GP) wordt hiervoor gebruikt omdat dit model niet alleen de verwachte prestaties van een specifieke configuratie voorspelt, maar ook de onzekerheidsmarge van die voorspelling kwantificeert.

Acquisitie Functie:

Op basis van de output van het surrogaat model gebruikt het algoritme een acquisitiefunctie bijvoorbeeld Expected Improvement of Upper Confidence Bound om te bepalen welke set gewichten in de volgende stap getest moet worden. Deze functie balanceert voortdurend tussen exploitatie waarbij configuraties worden verfijnd waarvan al bekend is dat ze goed presteren en exploratie waarbij nog onbekende parameter combinaties met een hoge onzekerheidsmarge worden getest omdat die mogelijk een beter globaal optimum bevatten .

Na elke nieuwe evaluatie in de effectieve pipeline wordt het resultaat gebruikt om het surrogaat model via een Bayesiaanse update bij te werken ook wel de posteriorverdeling genoemd. Door dit iteratieve leerproces convergeert het model efficiënt naar de optimale parameters configuratie.

Voordelen ten opzichte van grid search en random search

Ten opzichte van grid search:

Grid search is een exhaustieve brute-force methode die een vooraf gedefinieerde matrix van waarden doorloopt. Wanneer meerdere eigenschappen moeten worden gewogen ontstaat een combinatie explosie ook wel de curse of dimensionality genoemd. Grid search verspilt hierdoor rekenkracht aan evaluaties in parameter gebieden die duidelijk ondermaats presteren. Bayesian optimization vermijdt deze onnodige berekeningen door gericht te zoeken.

Ten opzichte van random search:

Hoewel random search beter schaalbaar is dan grid search doordat waarden willekeurig worden gekozen blijft het een ongeïnformeerde zoekmethode. Random search leert niet uit vorige iteraties. Bayesian optimization onderscheidt zich doordat historische evaluatie data worden gebruikt. Hierdoor kan het algoritme patronen uit eerdere tests herkennen en de volgende meest veelbelovende configuratie selecteren. Op die manier wordt het globale optimum in aanzienlijk minder stappen bereikt.

methodes

6. Data verzamelen

Binnen dit onderzoek worden drie verschillende soorten data gebruikt:

1. Technische tekeningen (DWG-PDF bestanden)
2. PDF-documentatie van componenten
3. Bedrijfsdocumenten (magazijn informatie)

6.1 Component Data uit DWG-bestanden

Er werd onderzocht hoe zoveel mogelijk data uit DWG-bestanden voornamelijk elektrische schema's kan worden gehaald. Hiervoor werd een eigen pipeline ontwikkeld.

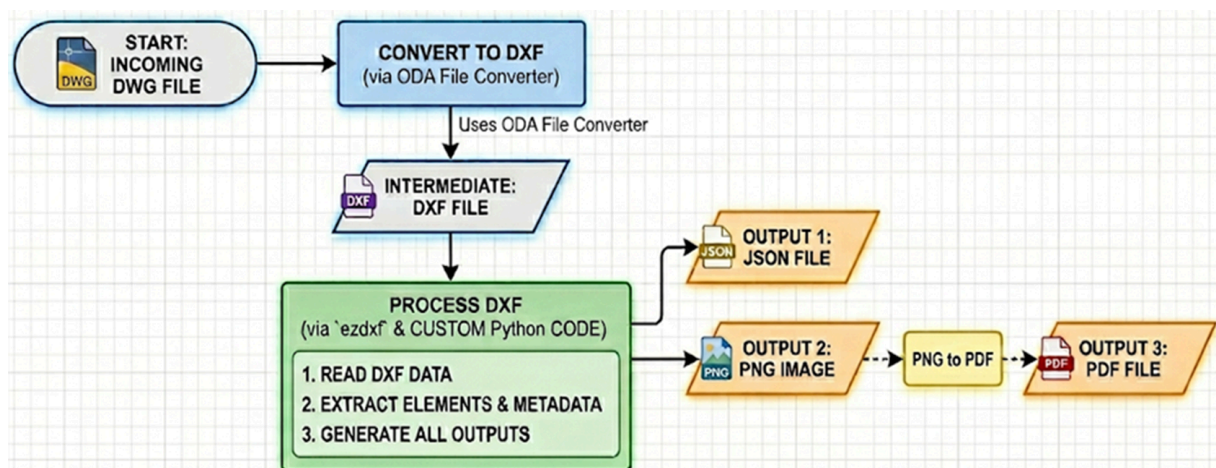


fig 3 DWG-extractie pipeline

De pipeline neemt een DWG-bestand als input. Via de ODA File Converter wordt dit bestand omgezet naar een DXF-formaat. Vervolgens wordt met een eigen ontwikkeld programma alle relevante informatie uit dit DXF-bestand geëxtraheerd en omgezet naar een JSON-bestand.

```

"component": "C0302022",
"location": [
  357.12,
  251.29
],
"attributes": [
  {
    "tag": "KLEMMR",
    "value": ""
  },
  {
    "tag": "LEVER",
    "value": ""
  },
  {
    "tag": "MERK",
    "value": ""
  },
  {
    "tag": "TYPE",
    "value": ""
  },
  {
    "tag": "NAAM",
    "value": ""
  },
  {
    "tag": "number of connectionPoints",
    "value": 1
  }
]

```

```

"component": "Ringcomp.",
"location": [
  211.91,
  67.08
],
"attributes": [
  {
    "tag": "TSTCODE",
    "value": "DMA"
  },
  {
    "tag": "PLCODE",
    "value": "504"
  },
  {
    "tag": "P",
    "value": "1.86 kW"
  },
  {
    "tag": "T",
    "value": "1480/min"
  },
  {
    "tag": "U",
    "value": "460 V"
  },
  {
    "tag": "I",
    "value": "3.10 A"
  }
]

```

fig 4 json tabel waar data niet is ingevuld fig 5 json tabel waar data wel is ingevuld

Daarnaast wordt er ook automatisch een PDF-versie van het bestand gegenereerd, aangezien het bedrijf niet voor elk DWG-bestand over een PDF-versie beschikt en omgekeerd.

Omdat het niet altijd duidelijk is waar specifieke componenten of elementen uit de JSON-structuur zich precies bevinden binnen de technische tekening werd een visualisatietool ontwikkeld die deze koppeling inzichtelijk maakt.

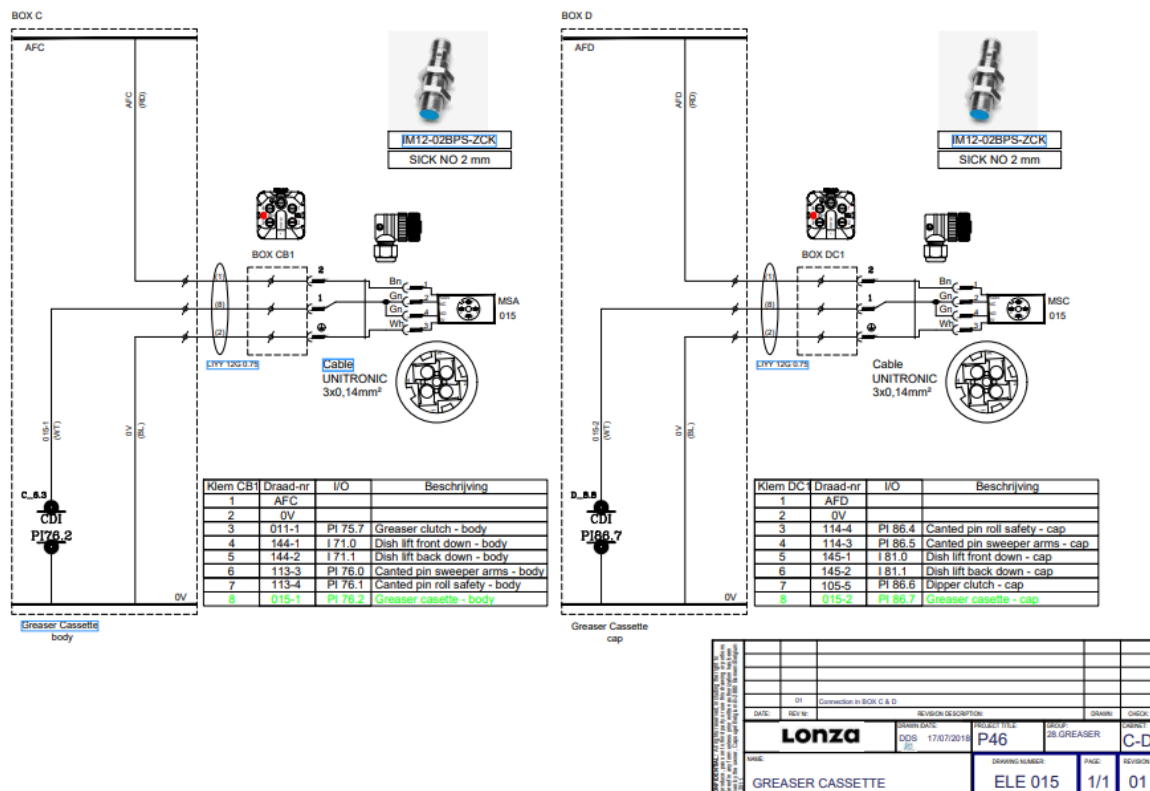


fig 7 visualisatie van herkende componenten door middel ocr

6.3 Verzamelen van onderdeel-documentatie

Aangezien niet voor elk onderdeel documentatie beschikbaar was, werd een web scraper ontwikkeld om deze informatie automatisch te verzamelen.

Als basis werd een Excel-bestand gebruikt dat alle componenten uit het magazijn bevat. Dit bestand omvat onder andere de naam, beschrijving, merk, artikelnummer, voorraad en locatie van elk component (naast bijkomende, minder relevante velden).

Op basis van deze data werden de componenten gegroepeerd per merk. Vervolgens werd een web-scraper ontwikkeld die voor elk onderdeel de bijhorende PDF-documentatie opzoekt, downloadt en opslaat in een gestructureerde map per merk.

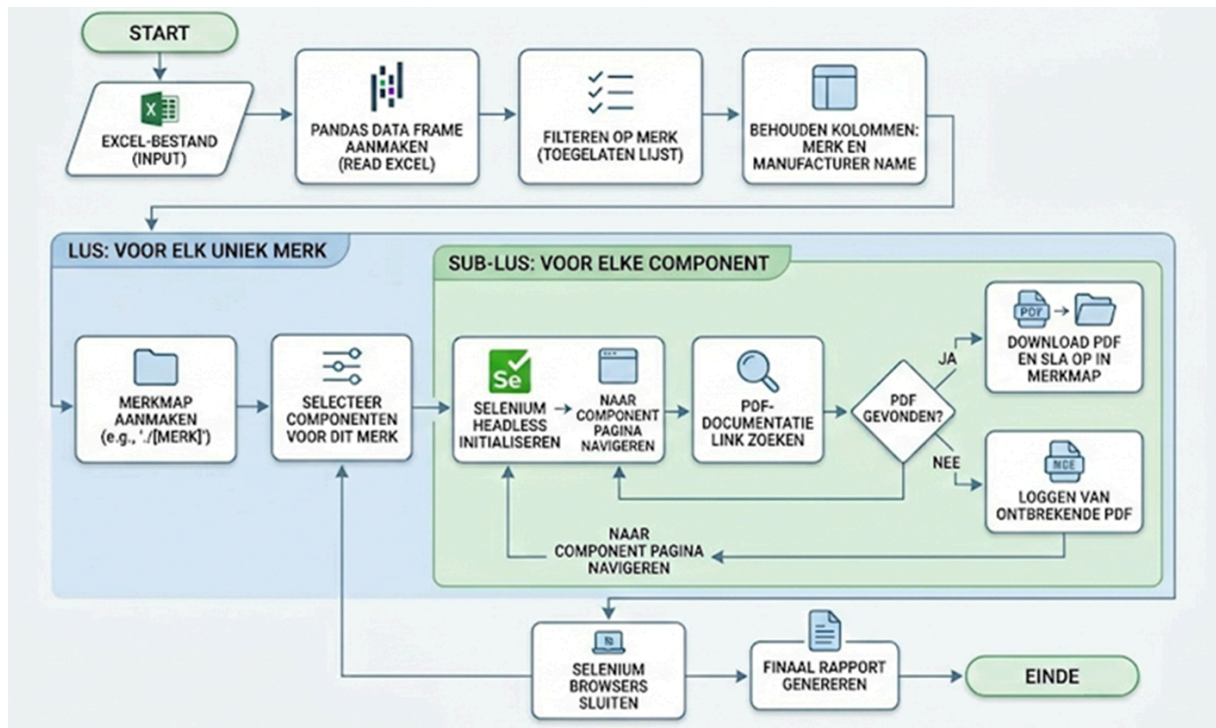


fig 8 pipeline om documenten te downloaden via een web scraper

7. Test methode en test sets

In dit hoofdstuk worden de methode en testsets toegelicht die gebruikt zijn voor het testen en valideren van de modellen, en hoe de test sets zijn samengesteld.

7.1 Magazijn-testset

De testset heeft de vorm van een JSON-bestand met dictionaries. Elke dictionary bevat de volgende twee keys:

- **text:** de OCR-text blob
- **sap nummer:** een uniek nummer dat aan elk onderdeel in het magazijn gekoppeld is

Dit SAP-nummer wordt gebruikt om magazijninformatie van een product op te halen, zoals locatie en stockinformatie. Elke onderdeelnode in de Neo4j-database bevat dit SAP-nummer als property.

Het doel van de testset is als volgt: een text blob wordt als input aan het model gegeven, waarna het model Neo4j-node-ID's terugstuurt. Vervolgens wordt binnen deze nodes gekeken naar de property *sap nummer* om te bepalen of de voorspelling correct is.

Eerst werd een handmatig samengestelde testset met ongeveer 80 testitems opgesteld (7.1.1). Later werd ook een synthetische testset ontwikkeld (7.1.2). Deze bevat meer dan 1000 testitems en kan eenvoudiger worden uitgebreid en aangepast.

Voor beide testsets wordt dezelfde evaluatiemethode gebruikt. Voor elke tekstblob wordt een vector similarity search uitgevoerd met een top-k waarde van 4. Dit levert vier onderdeelnodes op, gerangschikt van hoogste naar laagste zekerheid.

Vervolgens wordt gecontroleerd op welke positie het correcte SAP-nummer voorkomt. Wanneer het correcte SAP-nummer zich op de eerste positie bevindt, krijgt het datapunt een score van 1. Indien het correcte SAP-nummer op de tweede positie staat, krijgt het datapunt een score van 0,75. De derde positie krijgt een score van 0,5 en de vierde positie een score van 0,25. Wanneer het correcte SAP-nummer niet voorkomt binnen de vier opgehaalde nodes, krijgt het datapunt een score van 0.

Hierbij wordt de positie van het correcte resultaat binnen de top-k resultaten gebruikt om een gedeeltelijke score toe te kennen. Resultaten met een hogere zekerheid wegen daardoor zwaarder door dan resultaten met een lagere zekerheid. Een model kan dus nog steeds punten behalen wanneer het correcte antwoord niet op de eerste positie staat, zolang het correcte SAP-nummer binnen de opgehaalde top-k resultaten voorkomt.

Het scoresysteem is dynamisch en afhankelijk van de gekozen top-k waarde. Wanneer de top-k waarde bijvoorbeeld op 5 wordt ingesteld, bedraagt de score stap 0,2 in plaats van 0,25. Het systeem kan ook worden ingesteld op een top-k waarde van 1 om een zero-shot resultaat tussen 0 en 1 te verkrijgen. Dit werd toegepast in sectie (6.1.2).

7.1.1 De originele testset

Oorspronkelijk werd deze testset handmatig opgesteld door onderdelen in het magazijn te fotograferen en de bijhorende SAP-nummers te noteren. Op deze manier werden 82 onderdelen verzameld.

Later bleek deze aanpak echter een verspilling van tijd te zijn, aangezien het model slecht presteerde op deze testset. De behaalde scores lagen meestal tussen 48% en 60%, met een hoogste score van 63%.

Na verder onderzoek bleek het probleem niet bij het model te liggen, maar door de vervuilde testset.

Vaak werd het juiste onderdeel wel teruggevonden, maar kwam het SAP-nummer niet overeen met het SAP-nummer van het onderdeel in het magazijn.

1. Onderdelen werden niet altijd correct teruggeplaatst in het magazijn. Meerdere onderdelen werden teruggevonden op de verkeerde locatie, meestal één of twee vakken verwijderd van hun correcte plaats. resultaat was dat het model wel correct had maar dat het verkeerd was genoteerd in de testset
2. Tijdens het handmatig noteren van de SAP-nummers werden soms typfouten gemaakt.

Verder onderzoek naar de onderdelen die niet correct gevonden werden, toonde aan dat ongeveer 35% van de dataset vervuild was.

Daarnaast bood de originele testset van 82 items te weinig controle over specifieke variabelen en edge cases. Om deze redenen werd ervoor gekozen om een synthetische testset te maken waar volledige controle over was en waarvan gegarandeerd kon worden dat deze volledig zuiver is.

7.1.2 Synthetische testset

Zoals eerder besproken was de handgemaakte testset te klein, niet zuiver genoeg en was er onvoldoende controle. Het vervolg was daarom het maken van een synthetische dataset. Alle magazijn data staat in de Neo4j-nodes, waardoor de synthetische dataset als volgt kan worden opgebouwd.

Er wordt een Neo4j-node genomen. Deze bevat verschillende properties:

["Material Description", "Brand Name", "Product Name", "Manufacturer Part No"]

De vorm van deze testset is bijna hetzelfde als die van de handgemaakte testset: een JSON-bestand met dictionaries en twee keys: **ocr tekst** en **sap nummer**.

Enkel de OCR-text wordt nu gesimuleerd.

De properties kunnen uiteraard niet gewoon rechtstreeks aan elkaar geplakt worden en vervolgens als OCR-text beschouwd worden.

1. Een onderdeel bevat nooit perfect alle informatie.
2. Echte OCR is imperfect.
3. Anders zou een score van 100% behaald worden, aangezien alle informatie in de testset een directe kopie van de doel node is.

Dus hoe wordt OCR-text gesimuleerd?

Het proces werkt als volgt. Eerst wordt een doel node uit de database genomen. Dit is de node waarvoor de OCR-text gegenereerd wordt.

Er is een kans dat de Property *Brand Name* wordt toegevoegd. Dit op zichzelf is onvoldoende, dus er wordt altijd minstens één beschrijvende property toegevoegd uit:

["Material Description", "Product Name", "Manufacturer Part No"]

Daarna is er een kans van $1/x$ dat een volgende unieke property wordt toegevoegd uit:

["Material Description", "Brand Name", "Product Name", "Manufacturer Part No"]

Dit gebeurt in een loop voor X aantal keren.

Daarnaast is er ook een ruis kans van $1/x$ en een maximale ruis parameter.

Er is dus een kans van $1/x$ dat een property ruis krijgt. Dit betekent dat willekeurige karakters uit die property wegvallen of vervangen worden door random karakters. Dit dient om OCR-ruis te simuleren.

max-ruis % geeft aan hoeveel karakters in een tekst maximaal mogen veranderen of verdwijnen.

Bijvoorbeeld met een maximum van 20%:

“GS1-8712345-78145678”

Deze string bevat 20 karakters, waardoor maximaal 4 karakters gemuteerd verwijderd of toegevoegd mogen worden:

“O81-871234578146678”

Bij het creëren van de dataset zijn er dus meerdere variabelen die aangepast kunnen worden:

- max aantal items in de testset
- kans dat **Brand Name** voorkomt
- kans op een volgende property
- max aantal properties per item
- kans op ruis
- max ruis %

Het synthetisch genereren van de testset geeft de volgende voordelen:

1. Grote testsets kunnen snel en dynamisch gegenereerd worden.
2. Testsets kunnen gemakkelijk gegenereerd worden om verschillende scenario's te testen, zoals:
 - wat als er veel OCR-ruis is
 - wat als onderdelen weinig informatie bevatten
 - hoeveel invloed heeft de merknaam
3. Wanneer de merknaam gevonden wordt, wordt enkel verder gezocht binnen dit merk (zie hoofdstuk 8.2).
4. Met deze dynamische testset is het veel gemakkelijker en duidelijker om te zien waar het model goed mee presteert en waar het moeite mee heeft.

Flowchart-overzicht van het volledige proces. zie **figuur 9**

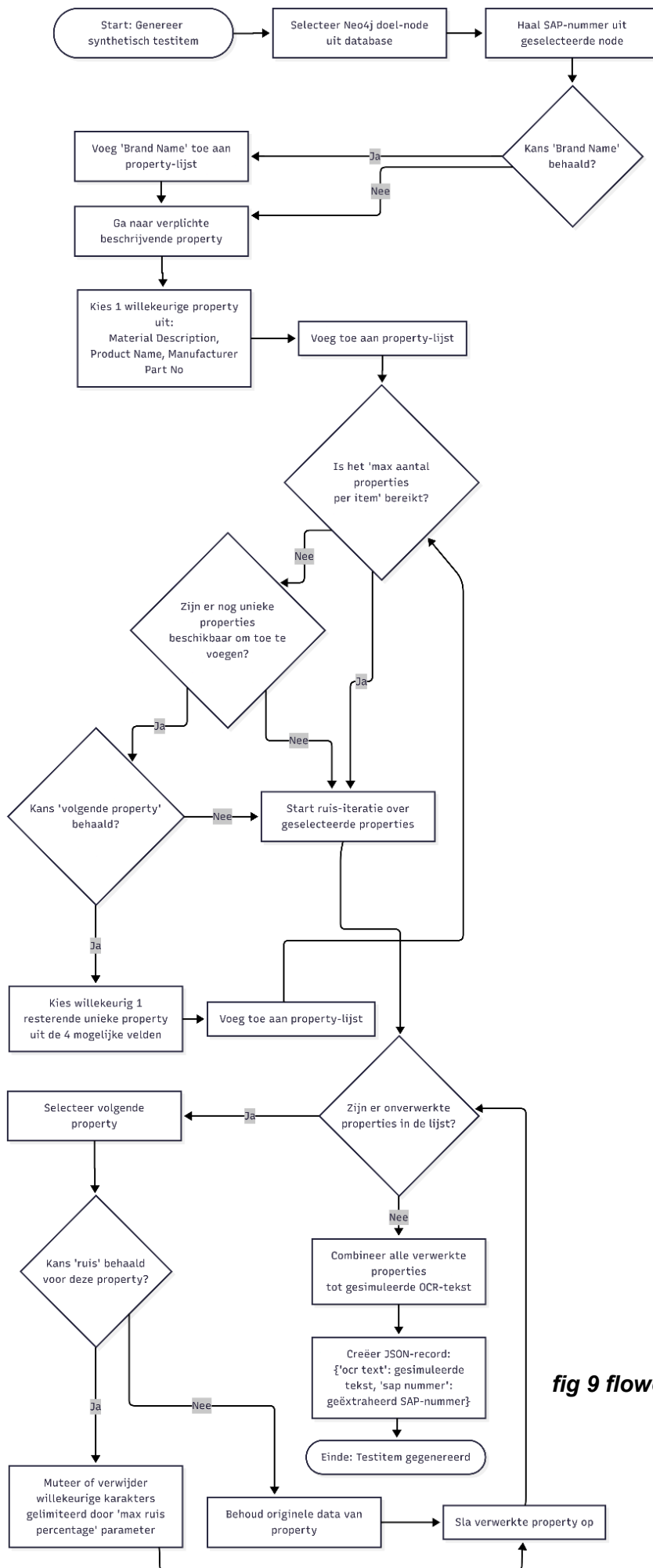


fig 9 flowchart Synthetische testset

7.1.3 Meerdere synthetische testsets / model evaluatie

Zoals eerder besproken, bieden synthetische testsets de mogelijkheid om parameters aan te passen zodat getest kan worden welke factoren de meeste invloed hebben op de resultaten en hoe robuust de modellen zijn.

In de resultaten van hoofdstuk 6 is een matrix weergegeven waarbij elke cel een aparte test set van 200 items voorstelt, opgebouwd uit een combinatie van de X- en Y-as als argumenten.

De Y-as toont hoeveel ruis aan een testset werd toegevoegd, samen met de waarde van het bijhorende argument. De X-as toont hoeveel data elk item in de testset gemiddeld bevat. Daarnaast is er ook een extra argument, "Material Description Only". Wanneer deze waarde is ingesteld, betekent dit dat elk item binnen die testset enkel de beschrijving als argument bevat.

Dit werd gebruikt om te evalueren hoe goed het systeem presteert wanneer er geen vaste koppeling aanwezig is met kenmerken zoals "Manufacturer Part No".

De confusion matrix zonder resultaten is hier weergegeven. In hoofdstuk 10 worden de resultaten van elk getest embedding model besproken.

8. Magazijn en ocr RAG pipe line

8.1 Doel

Een van de opdrachten van het bedrijf was om op basis van een foto van een onderdeel te bepalen waar dit onderdeel zich in het magazijn bevindt. Daarnaast was het de bedoeling om OCR te gebruiken om onderdelen uit technische schema's te extraheren en deze te koppelen aan onderdelen in het magazijn.

Aangezien de DWG-pipeline niet voldoende was (zie sectie 6.1 voor de testset), om de nieuwe methode te testen, wordt er gebruikgemaakt van de magazijn test set (zie sectie 7.1).

8.2 Aanpak

Een eerste aanpak bestond uit het gebruik van afbeeldingen/video herkenning om onderdelen terug te vinden . Dit werkte, maar was niet voldoende precies en er was geen geschikte dataset beschikbaar.(elk onderdeel zou ongeveer 1 min lang aan reference video moeten hebben) daarom is er besloten om over te stappen naar ocr aanpak

Er was een XLS-bestand beschikbaar met alle data uit het magazijn, waarbij elk onderdeel een aparte rij heeft . Dit bestand werd omgezet naar Neo4j-nodes, waarin de volgende properties worden opgeslagen:

["Material Description", "Brand Name", "Product Name", "Manufacturer Part No", "location code"]

Een eerste benadering bestond uit het samenvoegen van alle data in één tekstblok en het genereren van één master embedding die alle informatie bevat. Vervolgens werd OCR toegepast (8.2.1) om tekst van een onderdeel te extraheren. Deze tekst werd samengevoegd tot één string, waarna een query embedding werd gemaakt (8.2.4) en een vector similarity search (2.1.1) werd uitgevoerd op de master embedding.

Deze methode werkte, maar leidde tot de volgende problemen. Wanneer alle tekst wordt samengevoegd, gaat belangrijke informatie verloren in een grote hoeveelheid tekst. Bijvoorbeeld het onderdeelnummer. Hierdoor worden vaak componenten gevonden die in de buurt liggen, maar verschillen in bijvoorbeeld productnummer of enkele karakters. De data is aanwezig, maar er wordt niet specifiek op gefocust, doordat een algemene zoekopdracht wordt uitgevoerd over alle data tegelijk.

Om dit op te lossen wordt RRF toegepast (zie hoofdstuk 4) en wordt voor elke property een aparte embedding gebruikt in plaats van één master embedding.

Om de correcte top-k nodes op te halen, wordt een complexere pipeline gebruikt. Deze werkt als volgt.

We gebruiken OCR om de tekst uit een afbeelding op te halen. Bounding boxes die op dezelfde rij liggen en zich niet te ver van elkaar bevinden, worden samengevoegd zodat er geen context verloren gaat.

Vervolgens wordt voor elke bounding box(rij/zin) een aparte embedding gemaakt.

Daarna wordt voor elke bounding box een vector similarity search uitgevoerd op de property "Brand Name", waarbij top-k = 1 wordt gebruikt. Hierdoor wordt per bounding box enkel de meest waarschijnlijke brand name verkregen. De similarity score en de brand name worden opgeslagen als een tuple.

Vervolgens wordt gecontroleerd of er een brand name aanwezig is met een similarity score hoger dan een ingestelde threshold. Indien dit het geval is, wordt ervan uitgegaan dat de merknaam correct is. In de volgende fase worden alle nodes eruit gefilterd die niet tot dit merk behoren.

fase 2 Er wordt opnieuw een vector similarity search uitgevoerd, maar dit keer op gerichte properties, bijvoorbeeld "Manufacturer Part No". Hierbij worden meerdere nodes opgehaald, aangezien de eerste match niet altijd het beste is. In dit voorbeeld wordt top-k = 10 gebruikt per bounding box. Elke gevonden node wordt toegevoegd aan een centrale lijst per property.

Indien er bijvoorbeeld 10 bounding boxes zijn op een afbeelding en per bounding box 10 nodes worden opgehaald, ontstaat een lijst van 100 nodes per property. Dit leidt tot twee problemen:

1. De lijst bevat veel nodes met een lage similarity score.
2. Nodes komen meerdere keren voor.

Bijvoorbeeld: bounding box 1 vindt node 53 met een similarity score van 0,8, terwijl bounding box 2 dezelfde node opnieuw vindt met een score van 0,6.

Om dit op te lossen wordt een functie gebruikt genaamd *non_duplicate_max_pooling*. Deze functie verwijdert dubbele nodes en behoudt enkel de node met de hoogste similarity score.

De resulterende lijst van unieke nodes wordt gesorteerd op similarity score, waarna de top-k wordt geselecteerd. Dit resulteert in een lijst van de meest relevante nodes voor de betreffende property. Deze lijst wordt toegevoegd aan een centrale hoofdlijst.

Dit proces wordt herhaald voor al deze properties:

["Material Description", "Manufacturer Part No", "Product Name"]

Als eindresultaat krijgen we voor elke property een lijst met unieke nodes die het dichtst aansluiten bij de query.

Op deze lijsten wordt vervolgens weighted RRF toegepast om een verbeterde top-k te verkrijgen. Daarna worden de resultaten opnieuw gesorteerd op confidence. Deze methode levert een duidelijk beter resultaat op.

In de uiteindelijke pipeline zijn de optimale parameters echter niet gekend. Voor RRF wordt standaard een waarde van 60 gebruikt, maar het is onduidelijk of dit hier optimaal is. Daarnaast is niet gekend welke properties het zwaarst moeten doorwegen binnen weighted RRF.

Er wordt aangenomen dat “Manufacturer Part No” en “Product Name” meer gewicht moeten krijgen, omdat deze waarden betrouwbaarder zijn, maar dit kan niet met zekerheid bevestigd worden.

Om deze parameters te optimaliseren wordt Bayesian optimization toegepast (zie hoofdstuk 5).

De gebruikte configuratie (zie figuur 10) voor de resultaten hiervan zie sectie 9.1.3

```
# Bayesian Optimization Configuratie
opt_config = {
    "n_trials": 180,
    "bounds": {
        # de confidante trachhold waar een node aan moet voldoen om geselecteerd te worden
        "trashold": {"low": 0.2, "high": 0.95},
        # hoe veel noders worden ompgenen bij elke bouding box property cominatie
        "search_top_k": {"low": 5, "high": 150},
        # de return waarde van de non_duplicate_max_pooling functie, bepaalt hoeveel nodes er uiteindelijk in de rrf komen per product
        "retrive_top_k": {"low": 5, "high": 80},
        # de parameter voor de RRF (Reciprocal Rank Fusion) functie
        "rrf_k": {"low": 10, "high": 120},
        # gewichten rrf baald hoe zwaar een property meeegt in de uiteindelijke score
        "brand_filter": {"low": 0.6, "high": 0.95},
        "weight_desc": {"low": 0.1, "high": 3.0},
        "weight_part": {"low": 0.1, "high": 3.0},
        "weight_name": {"low": 0.1, "high": 3.0},
    }
}
```

fig 10 Bayesian optimization config voor rrf

hier is een flowchart van de uiteindelijke pipeline zie **fig 11**

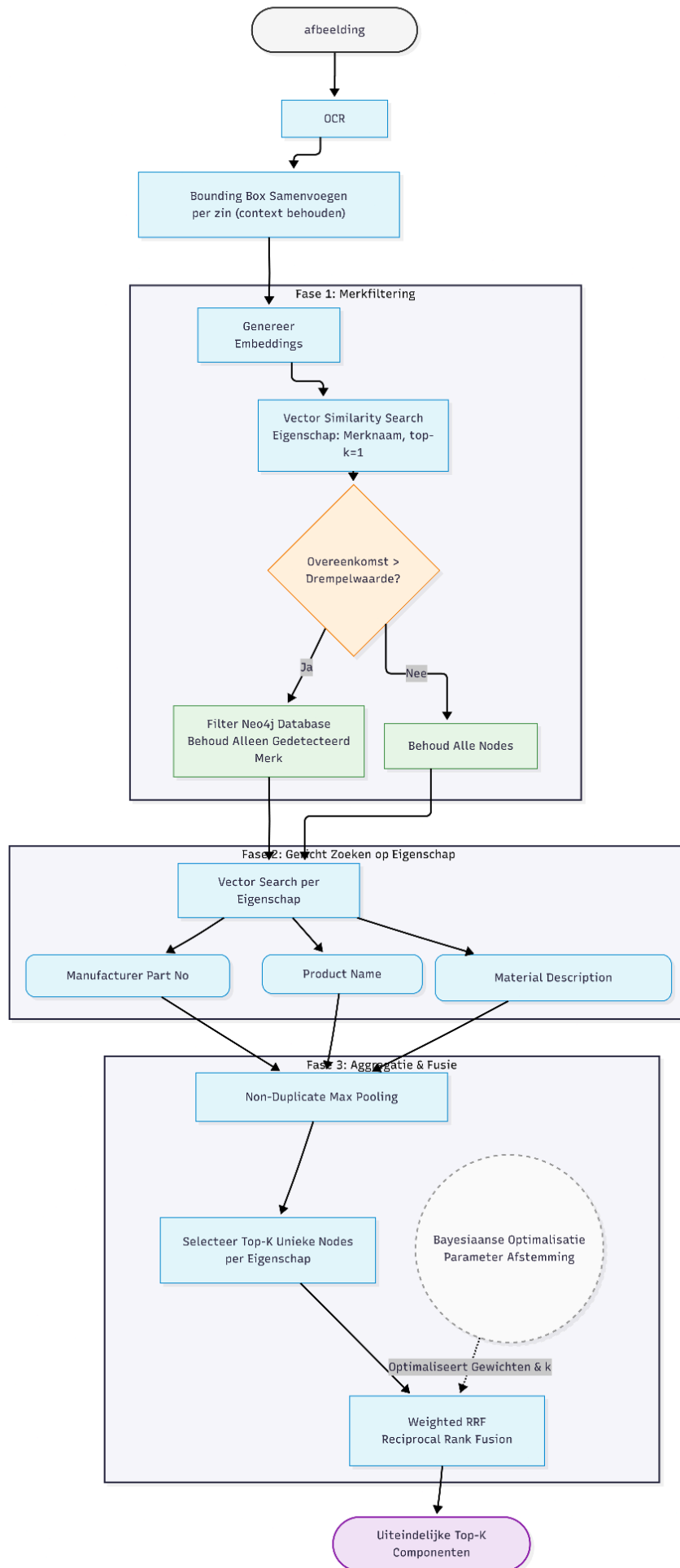


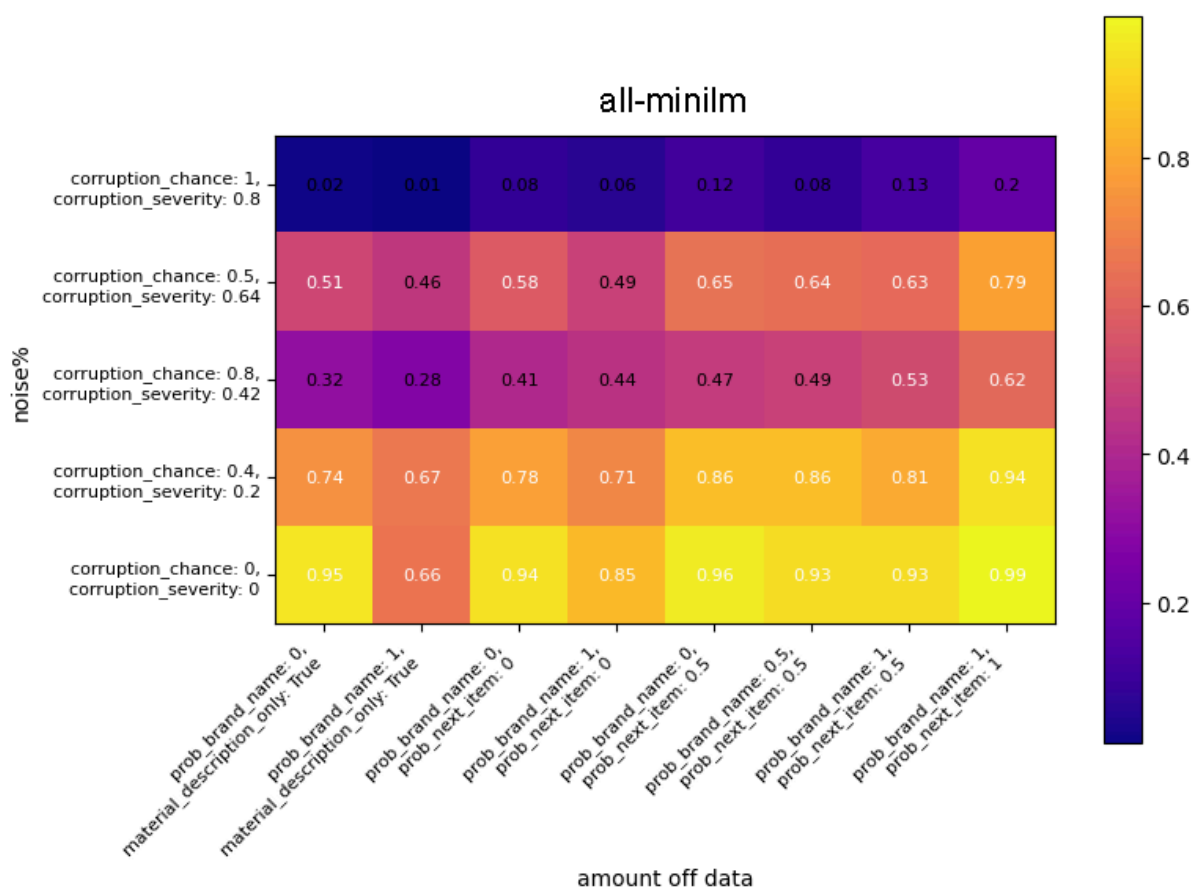
fig 11 ocr RRF flowchart

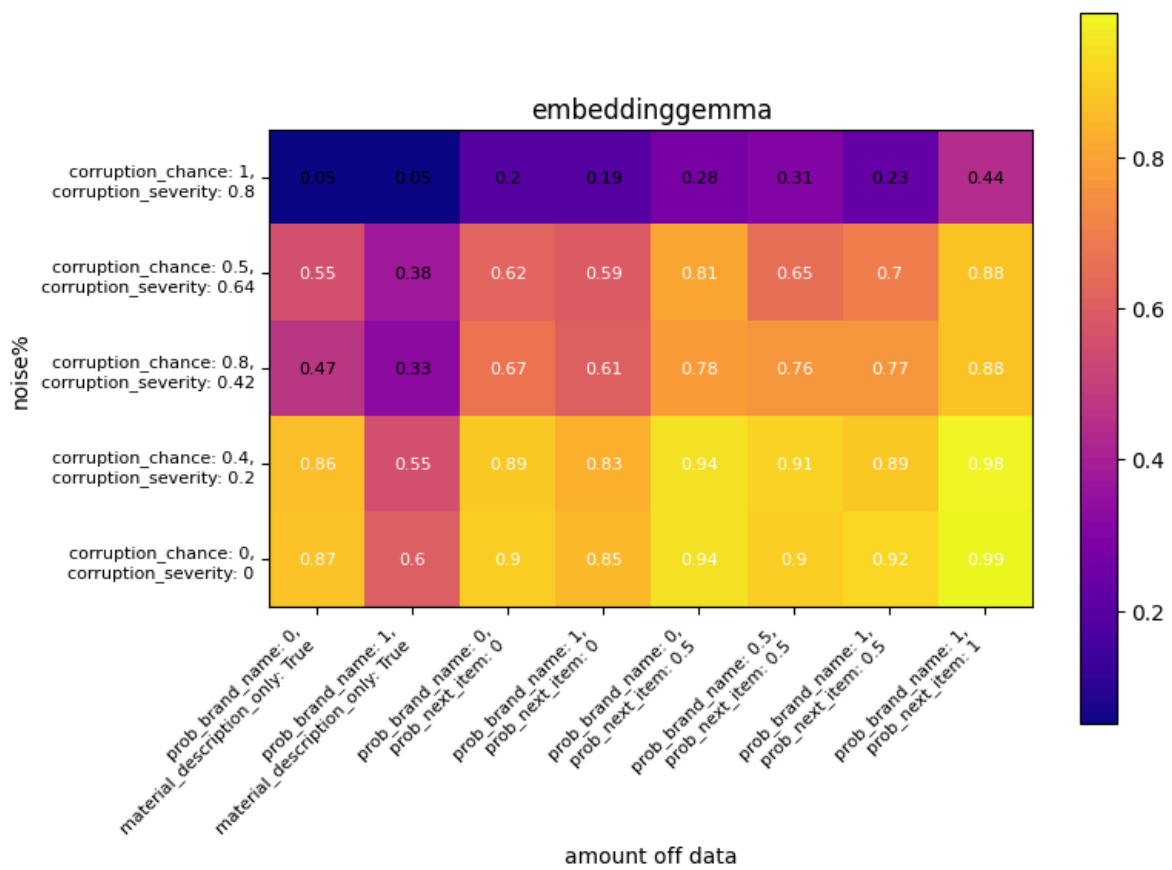
9. Resultaten

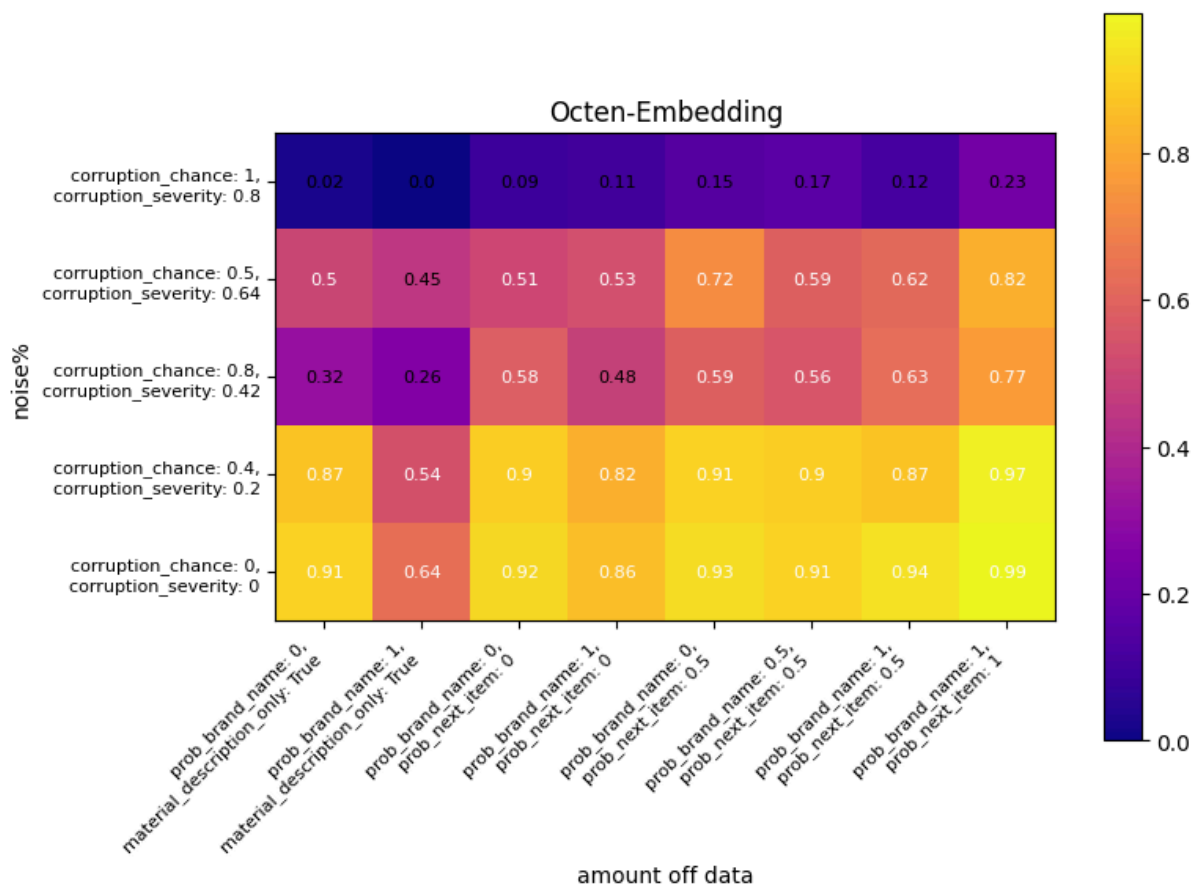
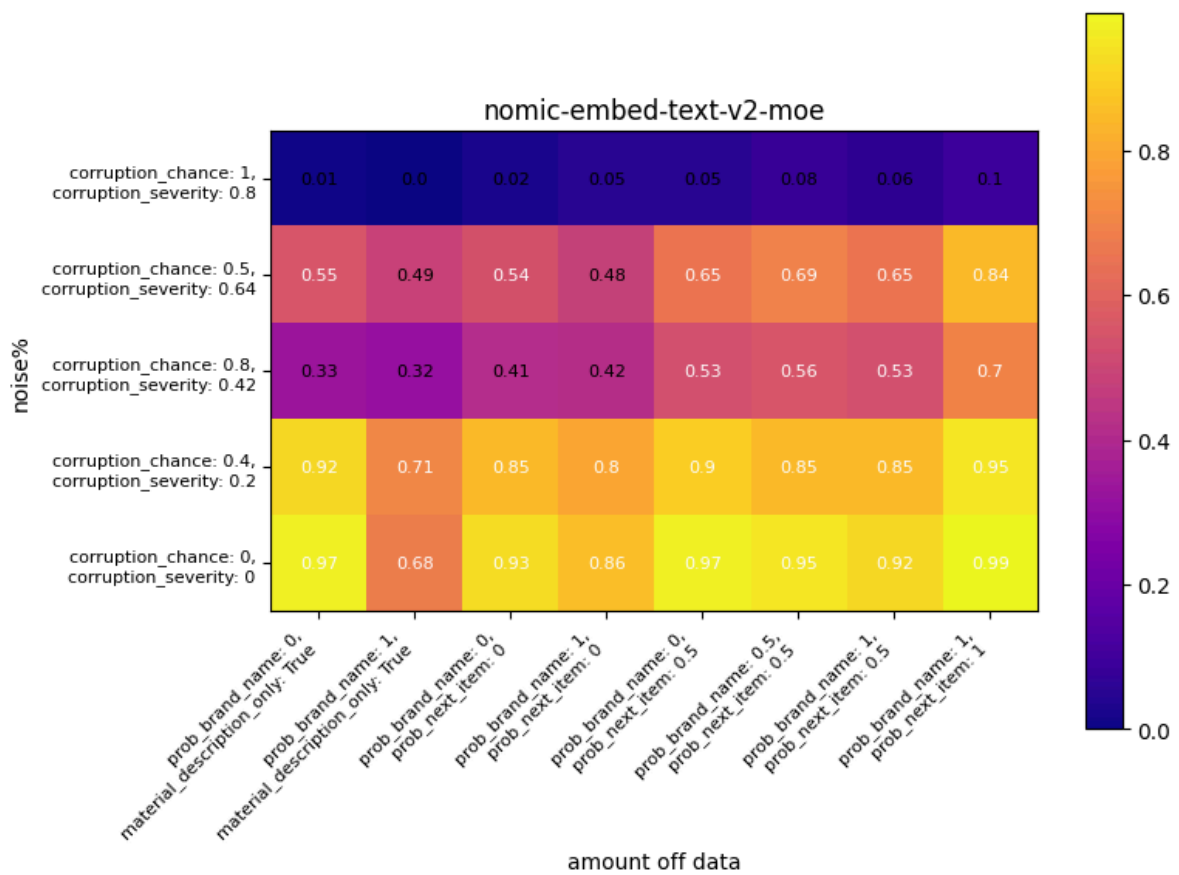
In dit hoofdstuk worden de resultaten van de geteste modellen en methodes besproken. In de vorige hoofdstukken werd uitgelegd hoe de testsets zijn opgesteld en welke methodes werden toegepast. In dit hoofdstuk wordt gekeken naar de resultaten en hoe goed elke methode en elk model scoort.

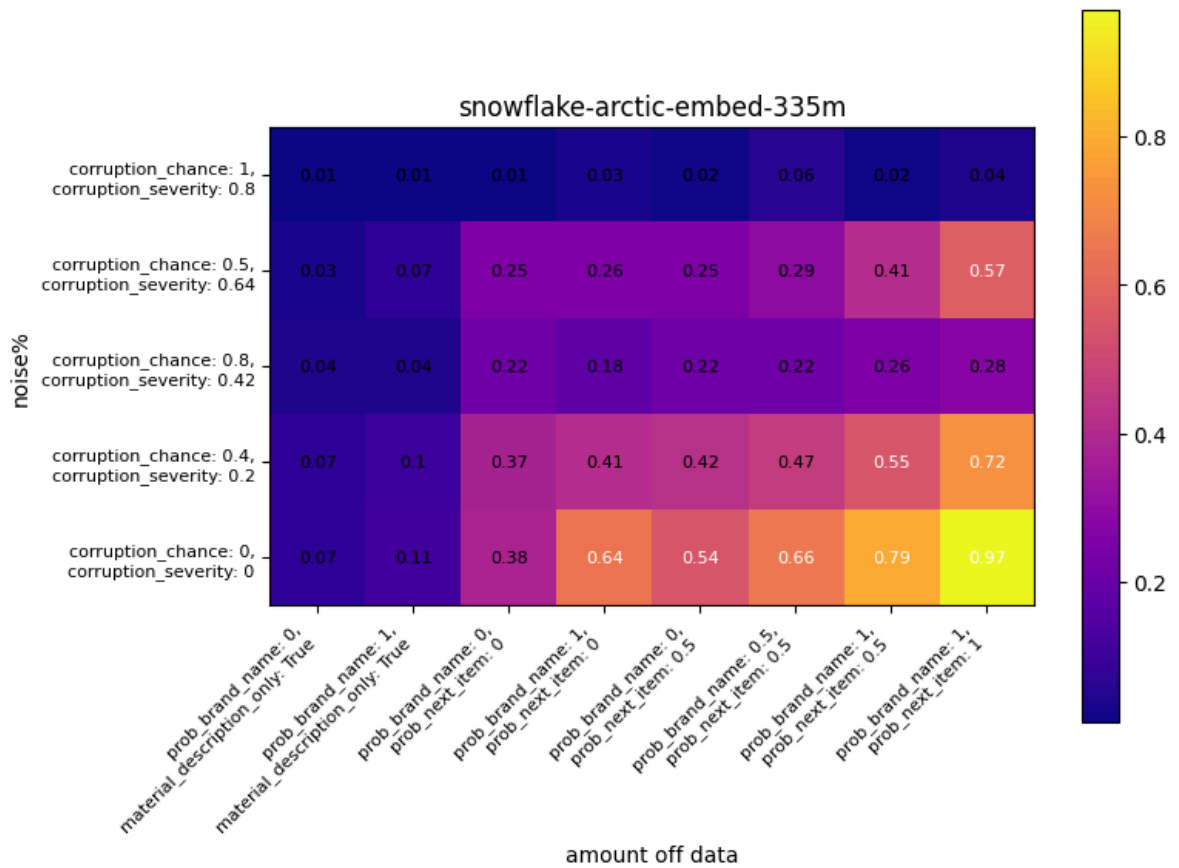
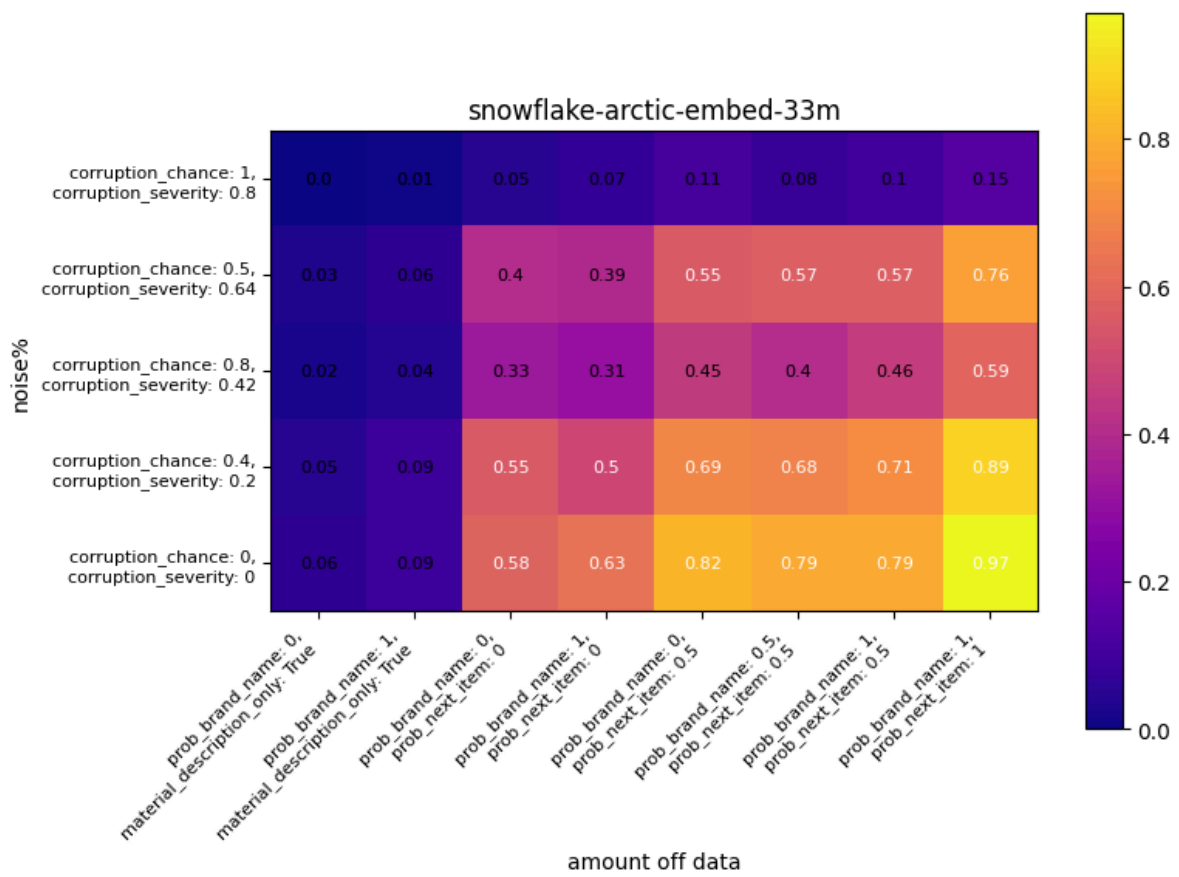
9.1 Embedding Modellen op de synthetische magazijn testsets

Dit zijn de resultaten van verschillende embedding modellen op verschillende gegenereerde testsets. Zoals besproken in hoofdstuk 3, blijven de RRF-waarden hetzelfde en zijn de testsets identiek voor elk model. Elk model haalt per item in de test 4 nodes op, waarbij de score afhangt van de positie indien de correcte node is gevonden ([zie hoofdstuk 3](#)).









9.1.1 evaluatie van de emding modellen

Zoals te zien is in de vorige resultaten presteren grotere modellen niet altijd beter op deze testsets. Vermoedelijk komt dit door de zoekmethode(RRF 6.3) die binnen deze test gebruikt wordt. In plaats van een algemene search wordt hier meer gebruikgemaakt van directe semantic search.

Grotere modellen zijn beter getraind om betekenissen en complexe verbanden uit grote tekstblokken te halen. Binnen deze testset is dit echter grotendeels overbodig, omdat gezocht wordt naar tekst die al zeer sterk overeenkomt met de zoekopdracht.

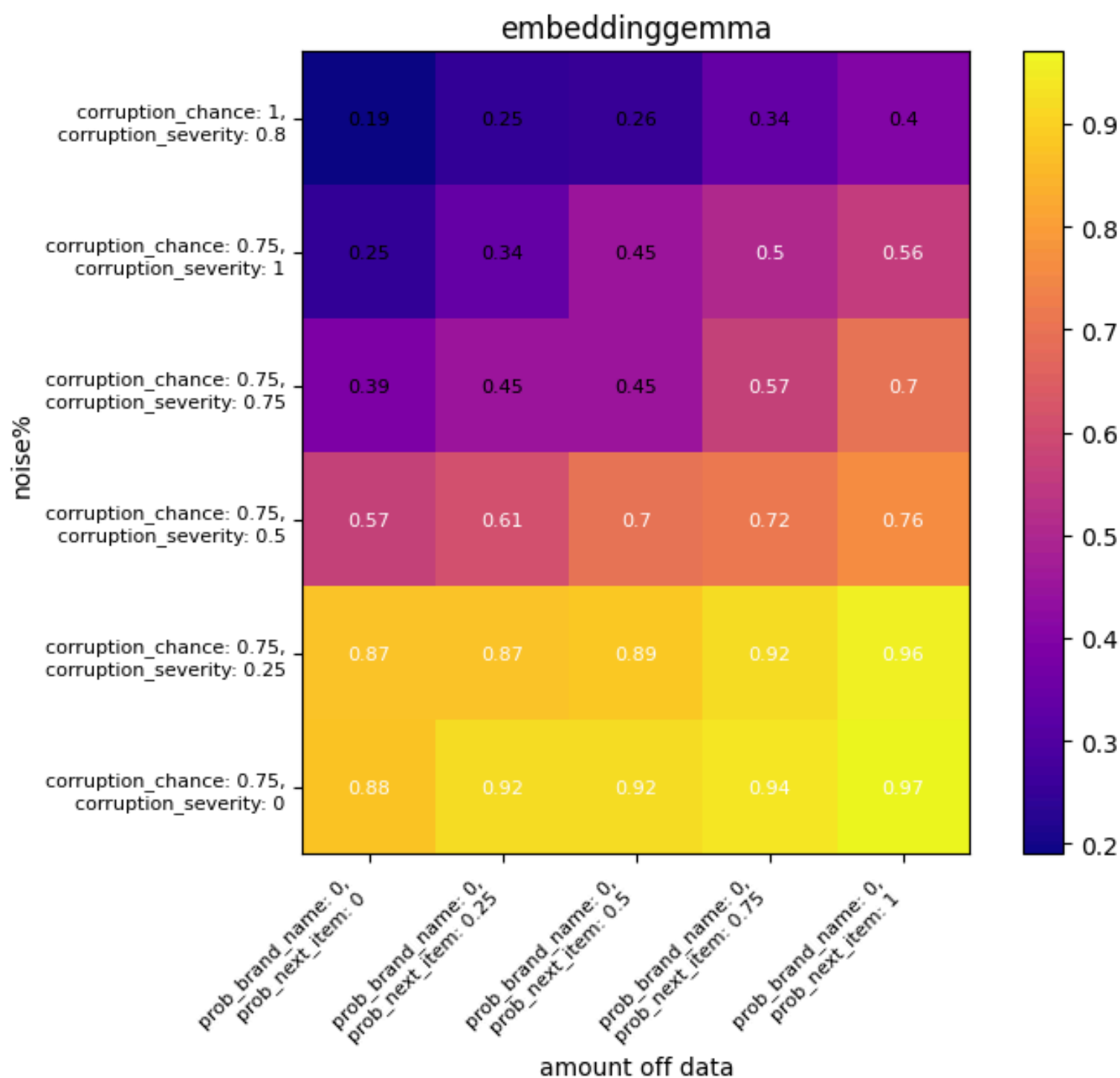
De resultaten tonen wel aan dat betere modellen veel beter kunnen omgaan met ruis wanneer er voldoende context aanwezig is. Dit is vooral zichtbaar in de rechterbovenhoek van de grafiek.

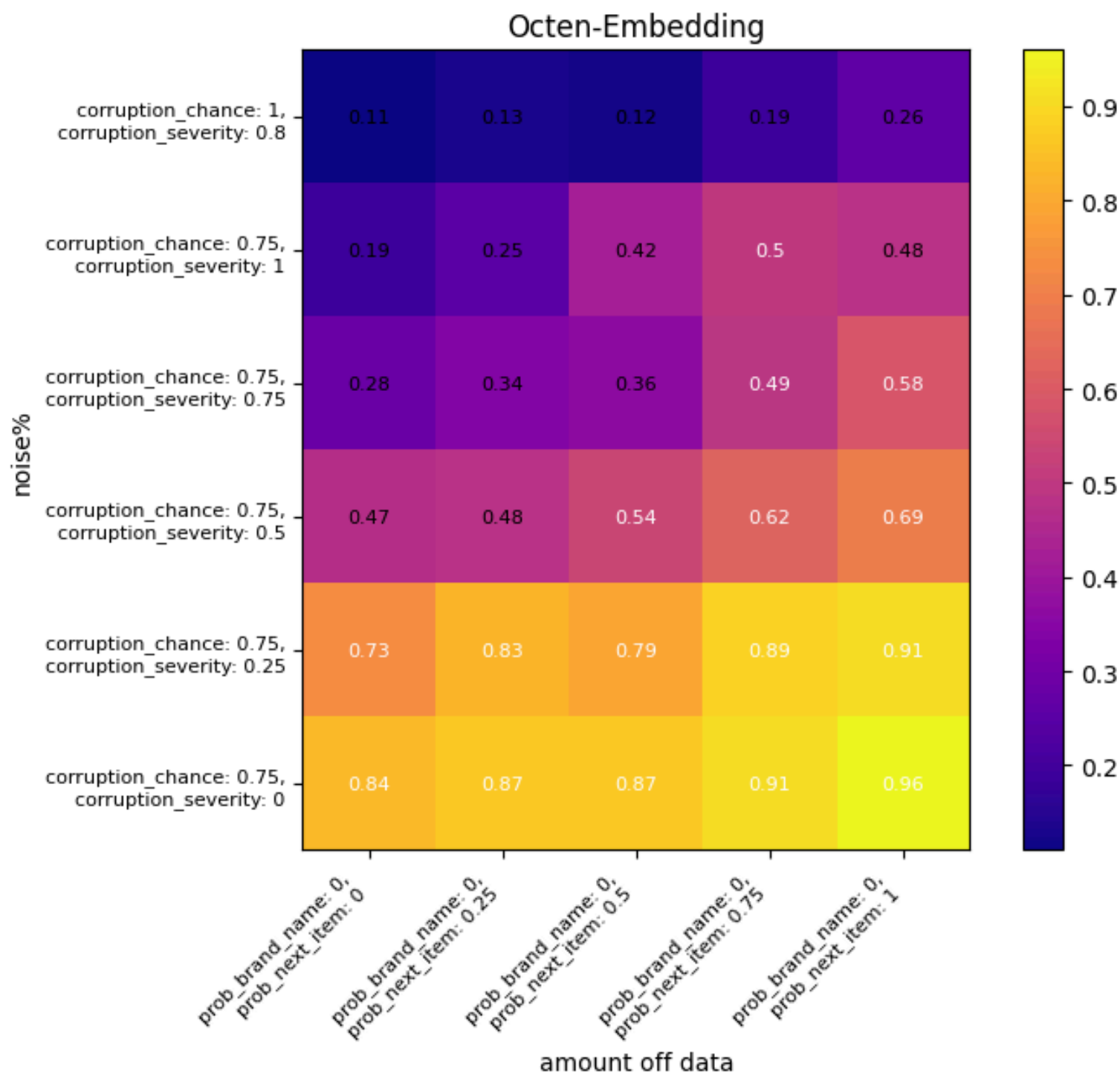
Daarnaast blijkt ook dat het oorspronkelijke idee om de KN te prunen op basis van de '*brand name*' geen goede resultaten opleverde. Wanneer een merknaam gedetecteerd werd, werd enkel verder gezocht binnen dat specifieke merk. Uit de resultaten blijkt echter dat de modellen slechter presteren wanneer de parameter "probably brand name" gelijk is aan 1. Dit betekent dat er een merknaam in de tekst aanwezig was en dat de zoekresultaten hierop gefilterd werden.

In eerste instantie werd verwacht dat het beperken van de zoekruimte tot een meer gefocuste sub-KN betere resultaten zou opleveren. De resultaten tonen echter het tegenovergestelde aan.

9.1.2 ruis robustness test

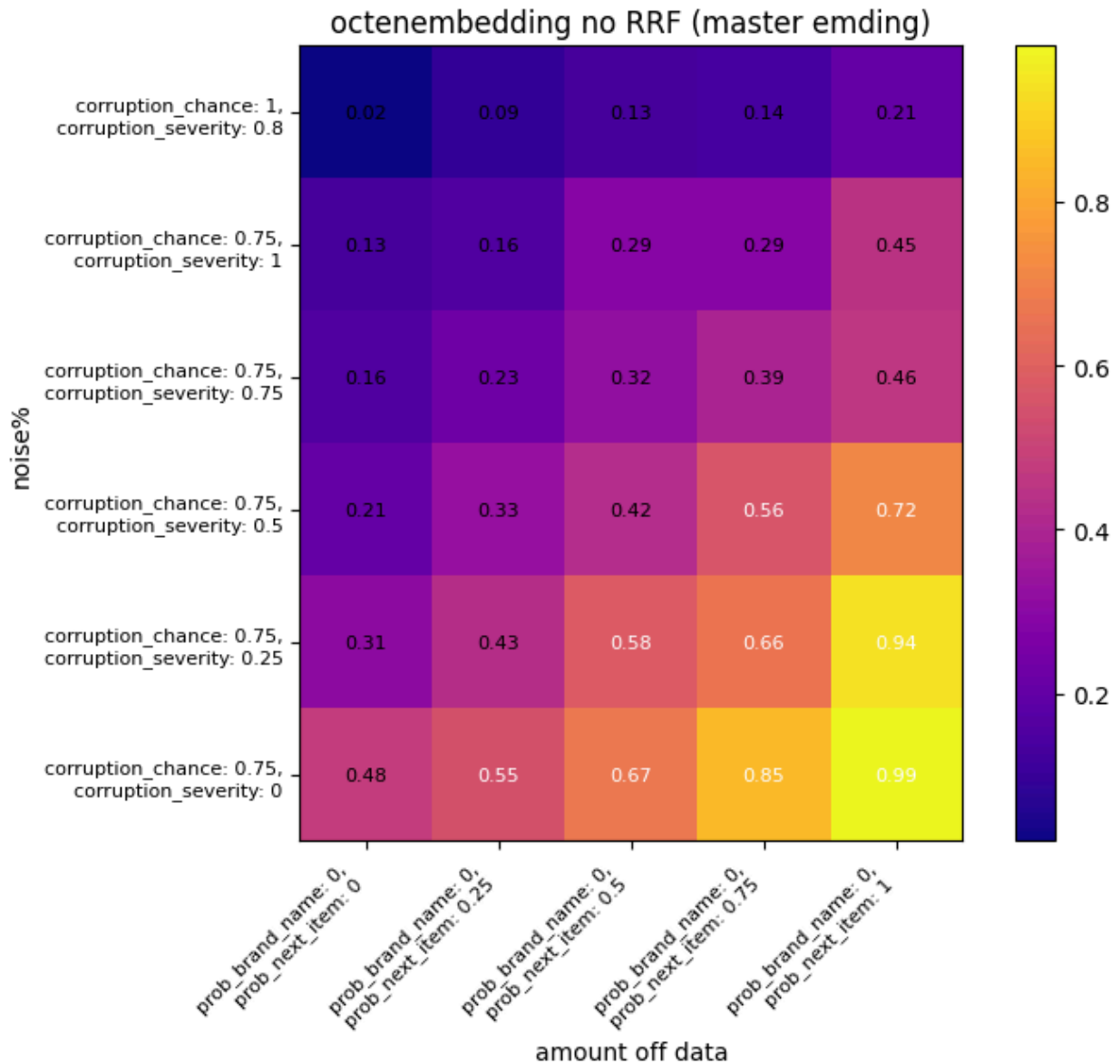
Om aan te tonen dat sommige modellen inderdaad beter bestand zijn tegen ruis is een scorematrix opgesteld waarin de merknaam volledig is weggelaten en diverse ruis parameters zijn toegevoegd. Er is duidelijk zichtbaar dat deze modellen, mits er voldoende tekstuele context aanwezig is, de juiste nodes terugvinden, zelfs bij aanzienlijke ruis. Tevens blijkt dat EmbeddingGamma vergelijkbare of zelfs betere prestaties levert dan een model dat zeven keer groter is (Octen-Embedding-8B).



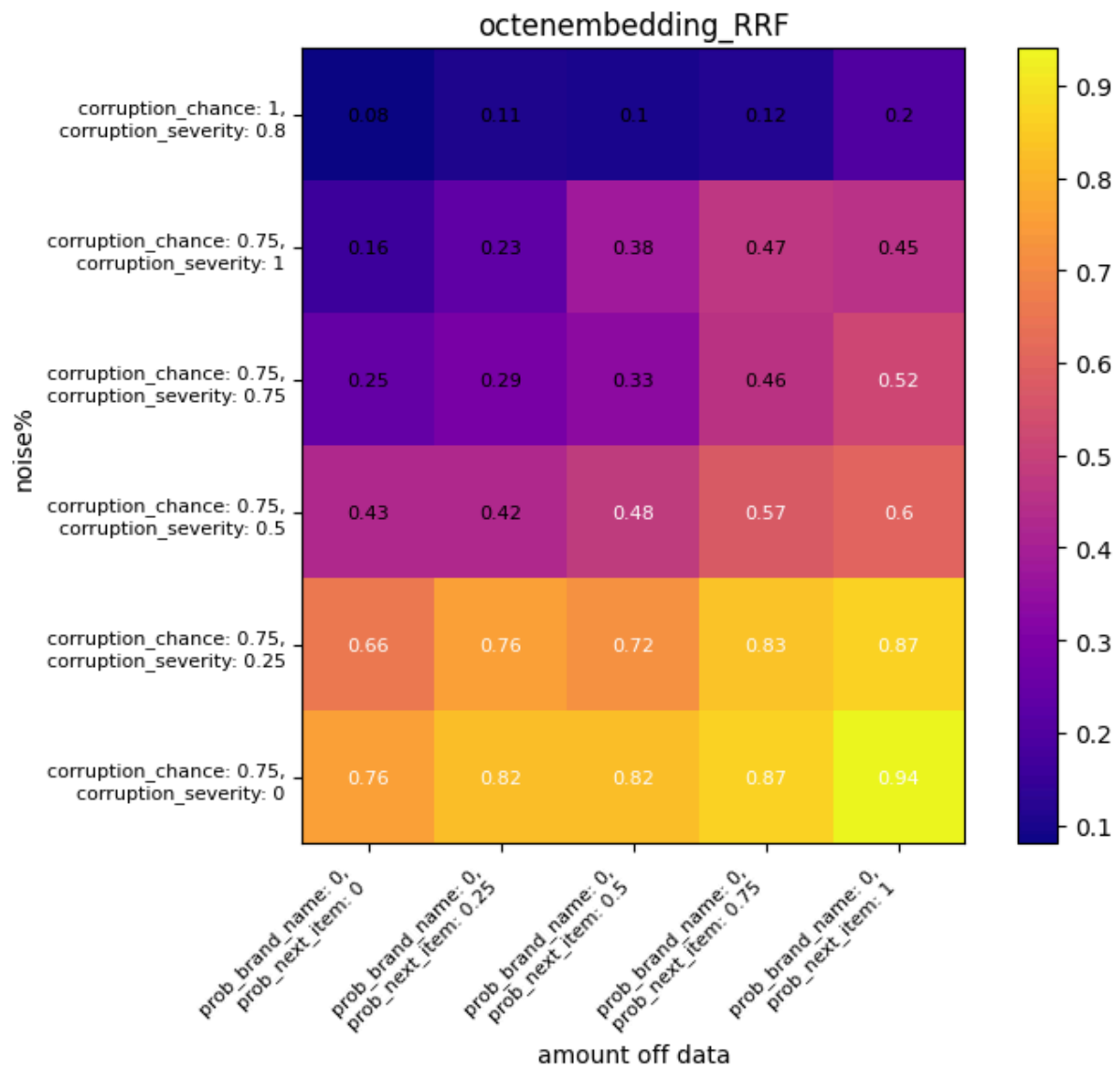


9.1.3 Weighted RRF

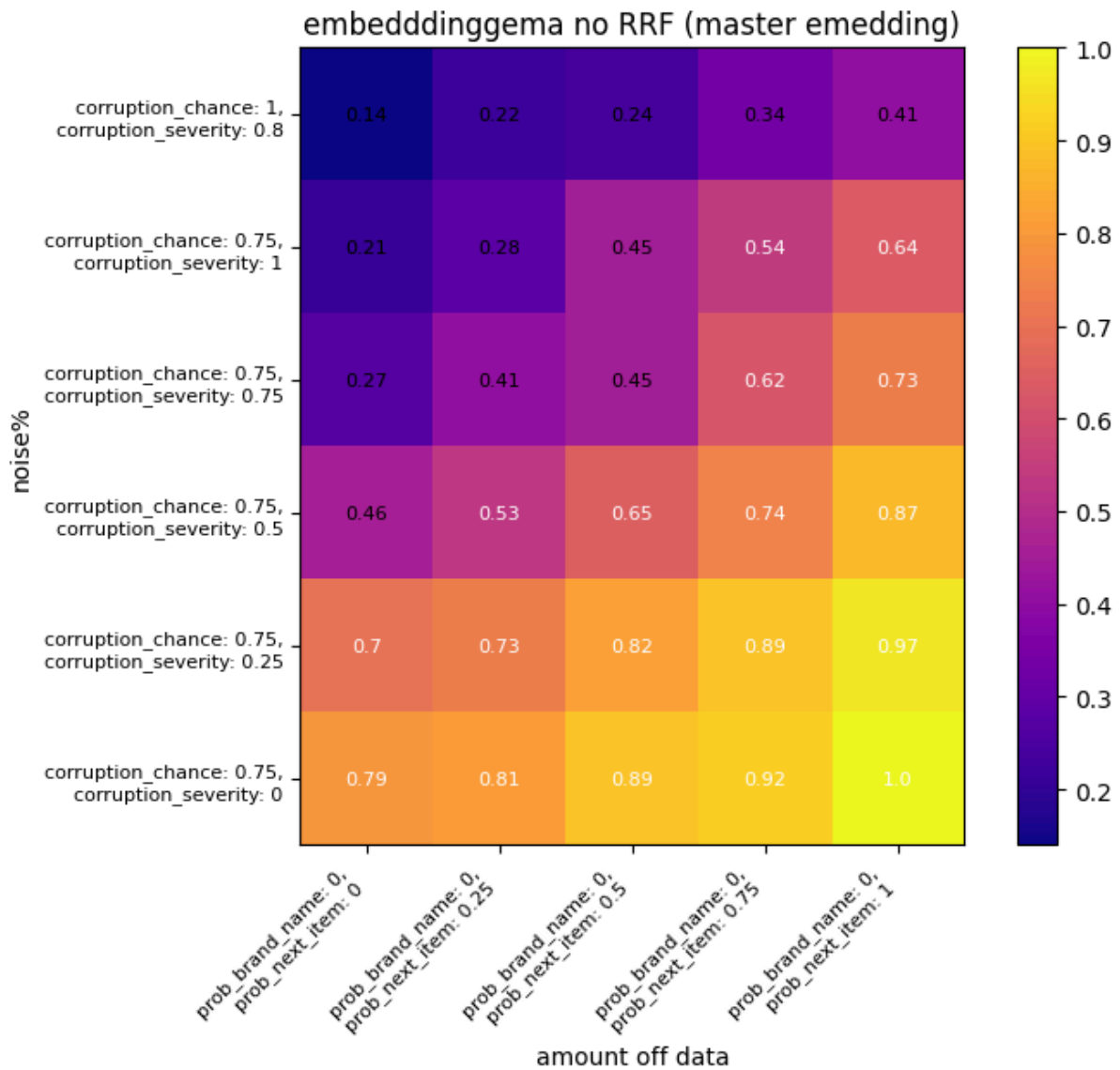
Hieronder worden de resultaten met en zonder RRF gepresenteerd. Bij deze testset is de scoreberekening aangepast: waar voorheen vier nodes werden opgehaald, wordt er nu gefocust op de best gevonden node om het effect van RRF beter te isoleren.



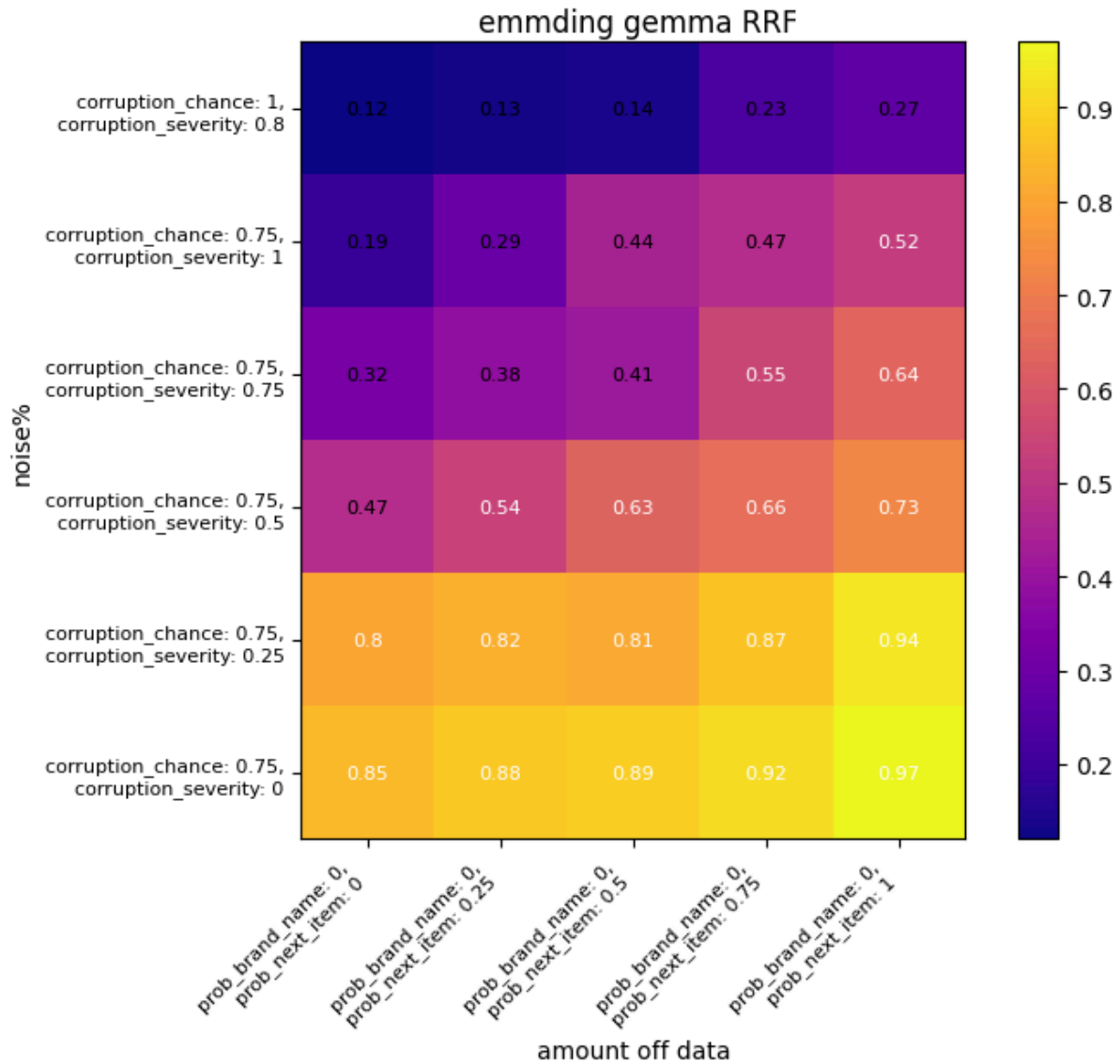
Dit zijn de resultaten wanneer een algemene embedding wordt gebruikt om alle properties op te slaan. Zoals links onderaan in de grafiek te zien is, scoorde dit slecht wanneer er weinig of zeer specifieke data aanwezig was. Een voorbeeld hiervan is wanneer enkel het onderdeelnummer in een item stond. Omdat het onderdeelnummer mee verwerkt wordt in de algemene master embedding, wordt het moeilijker om dit onderdeel correct terug te vinden.



Zoals hier te zien is, lost RRF dit probleem op doordat elke property een eigen embedding krijgt. Hierdoor presteren zoekopdrachten op specifieke properties beter.



Uit de tests is gebleken dat EmbeddingGamma zelfs zonder RRF beter presteert dan Octen-embedding met RRF. Bovendien is deze methode sneller vanwege de eenvoudige vector search.



Met RRF presteren de resultaten hier niet veel beter. De prestaties zijn licht verbeterd, maar het verschil is klein vergeleken met de andere embedding methode. Een nadeel dat hier zichtbaar is, is dat wanneer RRF wordt toegepast, het model een deel van zijn vermogen verliest om complexe relaties terug te vinden waardoor het iets slechter presteert in situaties met veel ruis.

Note: Voor deze resultaten met Embedding Gamma RRF werd

parameter optimalisatie() uitgevoerd om de beste score te verkrijgen. Deze optimalisatie werd uitgevoerd op een aparte dataset, zodat er geen informatie van de testset gebruikt werd tijdens het optimalisatieproces. De parameter optimalisatie verbeterde de score op de testset licht.

Het is wel interessant dat de optimale waarden voor de gevonden RRF-gewichten sterk verschillen van die van de andere embedding methode. Bij de andere methode lagen de gewichten voor bijna alle properties zeer dicht bij elkaar. Wanneer dezelfde optimalisatie wordt uitgevoerd met Embedding Gamma, zien de gewichten er echter volledig anders uit.

Octen Embedding RRF optimale gewichten :

weight_Material_Description: 1.3

weight_Manufacturer_Part_No: 1.47

weight_Product_Name: 1.48

Embedding gamma RRF optimale gewichten

weight_Material_Description: 2.7

weight_Manufacturer_Part_No: 1.4

weight_Product_Name: 1.88

Dit toont aan dat Embedding Gamma een voorkeur geeft aan complexere properties zoals *Material Description*. Vermoedelijk komt dit doordat de optimalisatie dataset ook ruis bevatten (zodat de optimalisatie rekening hield met verschillende mogelijke situaties.) Wanneer meer gewicht werd toegekend aan *Material Description* was Embedding Gamma beter in staat om moeilijkere items correct terug te vinden. Hierdoor behaalde het model hogere scores wanneer deze property een groter gewicht kreeg.

9.2 conclusie emmdig models en RRF

Hoewel RRF in het algemeen beter scoort, is het grootste nadeel dat er veel vector searches per bounding box property combinatie moeten worden uitgevoerd. Bij de master embedding wordt slechts één vector search uitgevoerd, wat direct resultaat oplevert. Bovendien is EmbeddingGamma aanzienlijk lichter en zijn de prestaties nagenoeg gelijk, ongeacht of RRF wordt toegepast.

Daarom is ervoor gekozen om in het eindproduct standaard de master embedding met embeddinggamma te gebruiken omdat deze sneller is en in de meeste gevallen voldoende presteert. In de UI is wel een checkbox voorzien waarmee de gebruiker alsnog RRF kan inschakelen indien gewenst.

10. Overzicht project en conclusie

In dit laatste hoofdstuk wordt de werking van het eindproduct besproken hoe de UI eruit ziet en welke doelstellingen wel en niet behaald zijn, en wat er veranderd is tegenover het originele opstart verslag.

10.1 user interface

Hieronder volgt een toelichting op de gebruikersinterface en de werking voor de eindgebruiker.

10.1.1 zoeken op foto

De camera kan worden gebruikt om tekst op te halen met OCR

(de gebruiker kan ook de gevonden tekst aanpassen of rechtstreeks op text zoeken)

Daarna kan op de “process”-knop worden geklikt. Dit geeft de gevonden onderdelen terug. Onder de process-knop wordt een dynamische lijst getoond met de gevonden onderdelen, waarbij de volgorde gebaseerd is op hoe zeker het model is dat het juiste onderdeel werd

teruggevonden.

Als test is hier enkel het onderdeelnummer ingegeven, dat met een hoge zekerheid wordt teruggevonden. In dat geval staat er slechts één item in de lijst, in dit geval *SIMATIC S7-1200* Indien het model minder zeker is dat het correcte onderdeel werd opgehaald, of indien meerdere nodes dicht bij elkaar liggen, worden er dynamisch meer onderdelen getoond. Zo wordt ervoor gezorgd dat de gebruiker altijd het juiste onderdeel kan terugvinden.

De lijst is gesorteerd van meest naar minst zeker en kan maximaal vijf resultaten tonen.fig 12

Door op een onderdeel in de lijst te klikken, wordt de gebruiker doorgeleid naar de informatiepagina.

Op de informatiepagina is alle relevante data van het betreffende onderdeel zichtbaar (fig 14).

(10.2)

Er is ook een debug-menu aanwezig, voornamelijk bedoeld voor testdoeleinden. Het genereert een logbestand met alle informatie over tekst en teruggevonden nodes (fig 13)

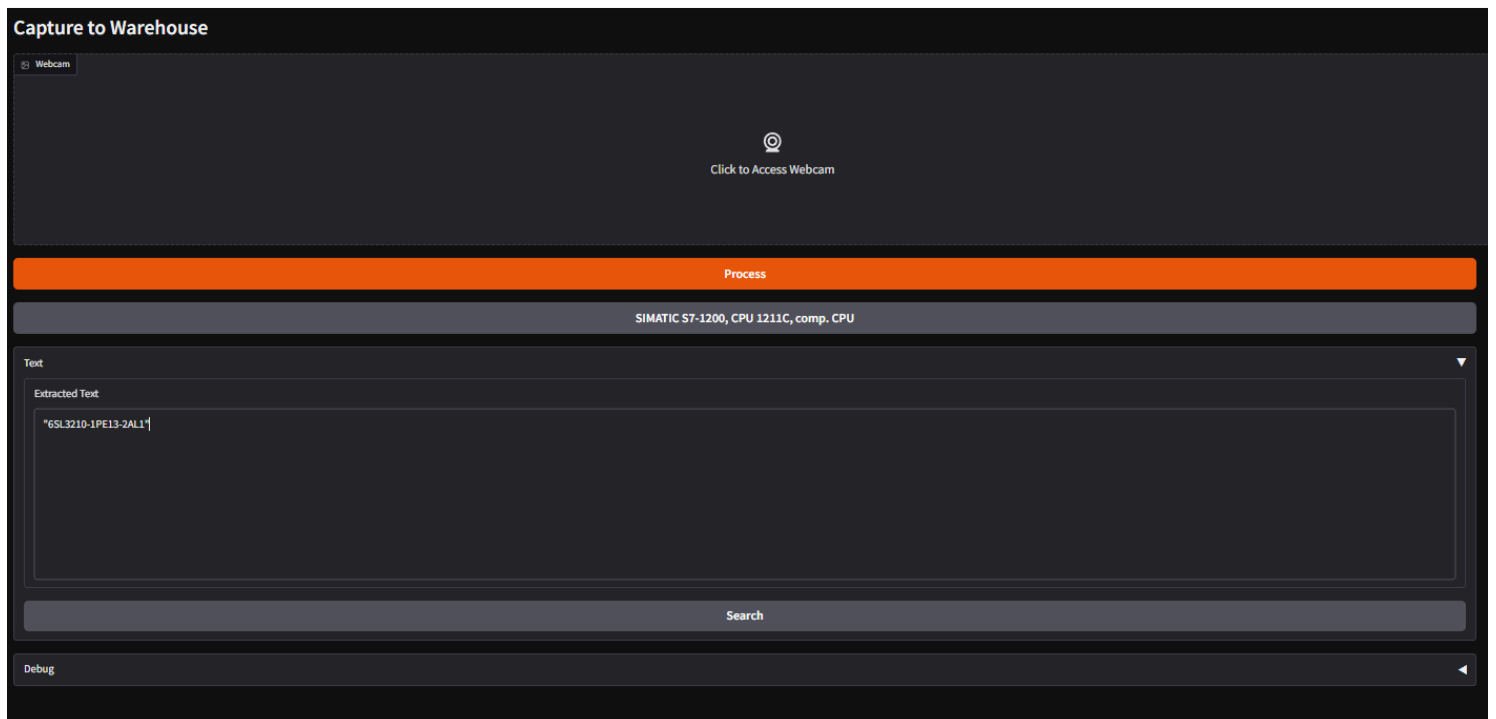


fig 12 pagina om onderdelen terug te vinden vanuit een foto

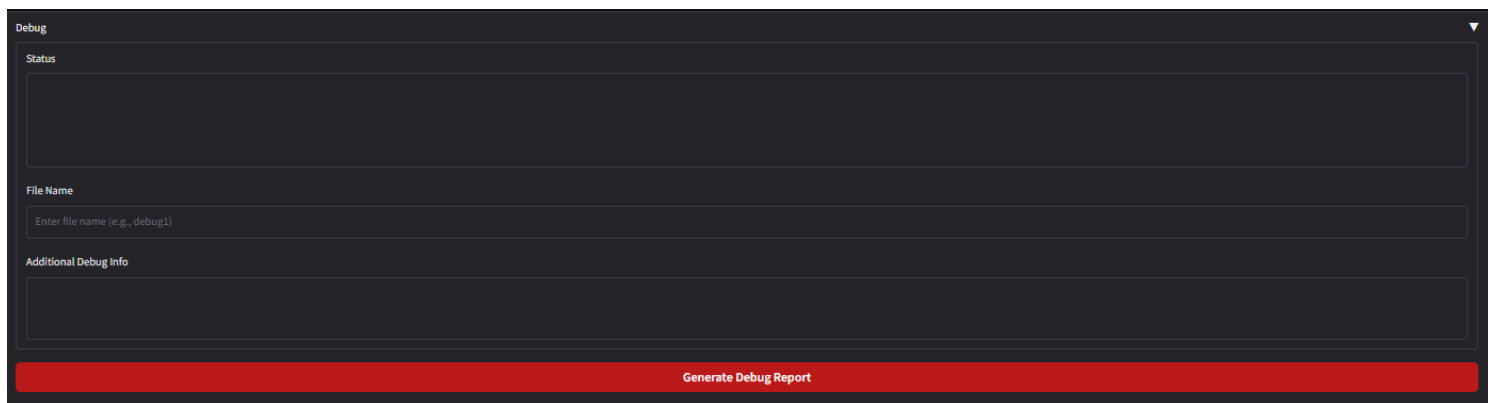


fig 13 debug menu

10.1.2 infopagina

Op de informatiepagina wordt gedetailleerde informatie over het geselecteerde product getoond (fig 14).

product naam, merk, magazijn voorraad ect

er zijn ook tabs voor magazijnlocatie (*fig 15*)

De magazijnlocatie van het onderdeel wordt visueel weergegeven.

het gele vak met een rode rand

en er is een tab om de documentatie van het onderdeel te bekijken(*fig 16*)

Via de knop 'Stel een vraag' kan de gebruiker naar de chatbot-pagina gaan om vragen over het document te stellen (10.1.3).

Product Info

Merke: SIEMENS

Beschrijving: SINAMICS POWER MODULE 6.75KW PM240-2

Magazijn locatie: E053V03

SAP nummer: 7018826

In voorraad: 2

▼ Extra details

IDc: 1183

Product naam: GSL3210-1PE13-2AL1

Manufacturer:

Manufacturer part number: GSL3210-1PE13-2AL1

Site: BEAB

State:

Oud magazijn nummer: 024720-00

Magazijn

Documentatie

fig 14 pagina waar de gebruiker info kan zien van een opgehaald product

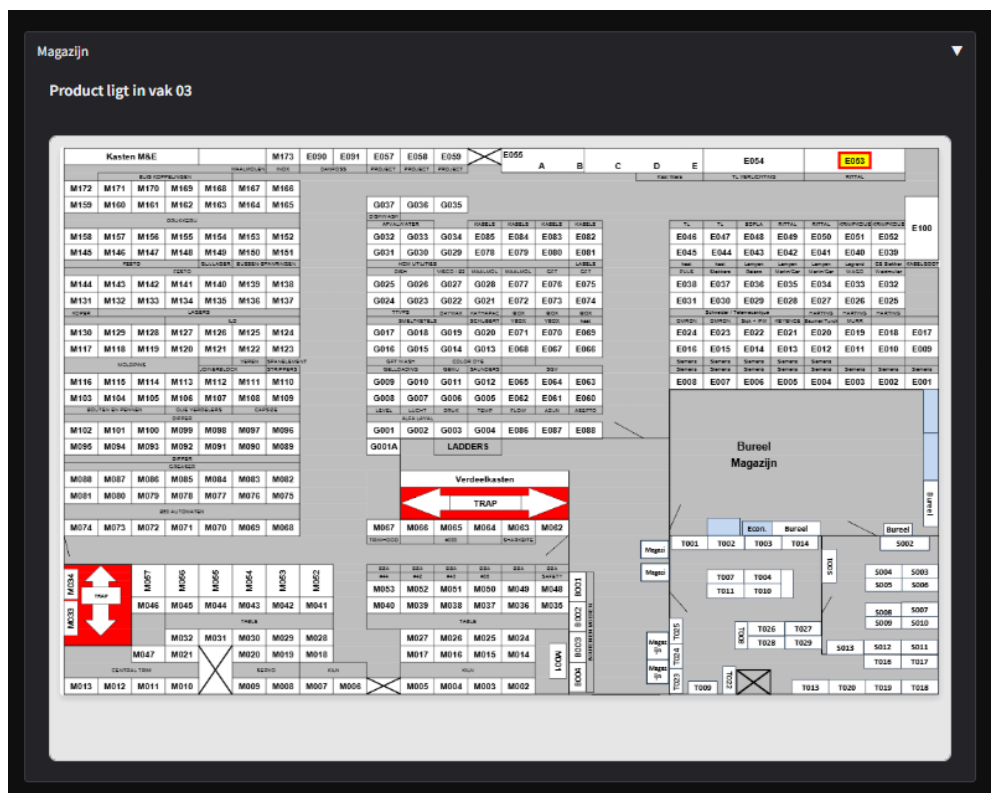


fig 15 magazijn layout waarbij de component locatie is aangeduid

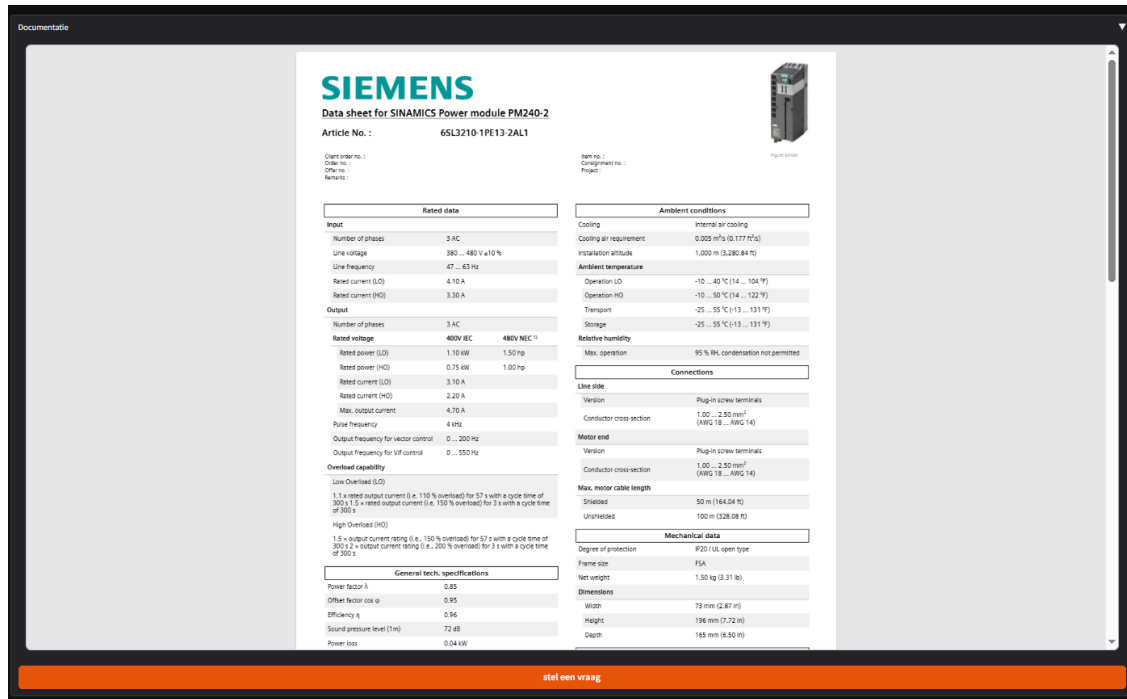


fig 16 documentatie preview

10.1.3 chatbot pagina

Op deze pagina kan interactie plaatsvinden met een chatbot voor vragen over geselecteerde onderdelen (fig 17). Er zijn drie hoofdelementen: de tab context (fig 17), de tab advanced settings (fig 18) en de chat interface zelf (fig 17).

en het chat interface zelf (fig 17)

de tab context toont over welke onderdelen het model info heeft

De vector search wordt beperkt tot een sub-KN met context over de relevante onderdelen, wat de nauwkeurigheid van de opgehaalde informatie verbetert.

In de chat interface kunnen vragen worden gesteld, waarbij de redenering, de resultaten van de vector search en debug-informatie desgewenst getoond kunnen worden (fig 21).

In de tab advanced settings (fig 18) kunnen instellingen zoals het modeltype en het systeem prompt worden aangepast (fig 19).

Model parameters zoals temperatuur, top K en het aantal op te halen items bij vector search (Top K) zijn eveneens configureerbaar (fig 20).

dan is er nog een tab met extra opties (fig 22)

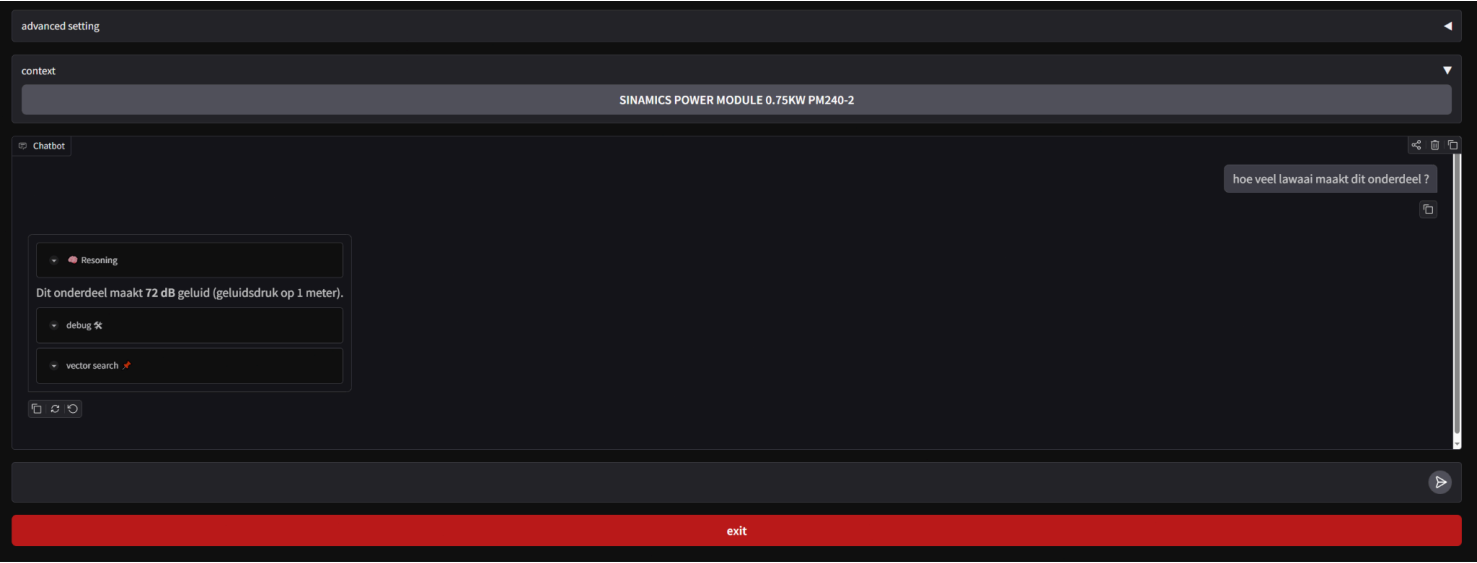


fig 17 chatbot interface

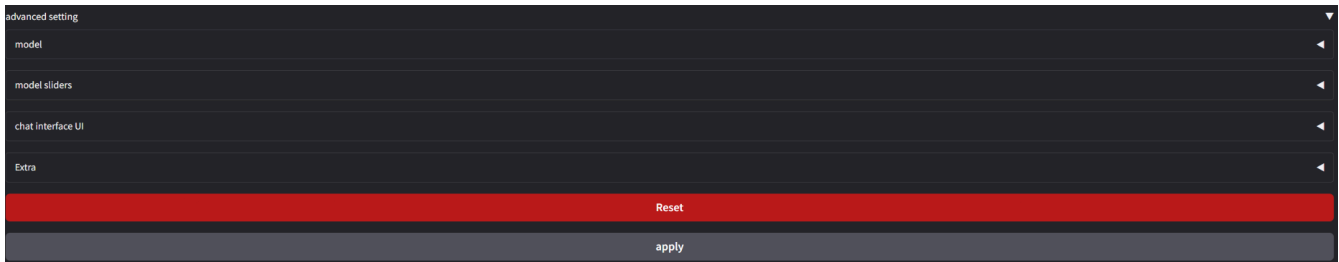


fig 18 instellingenmenu

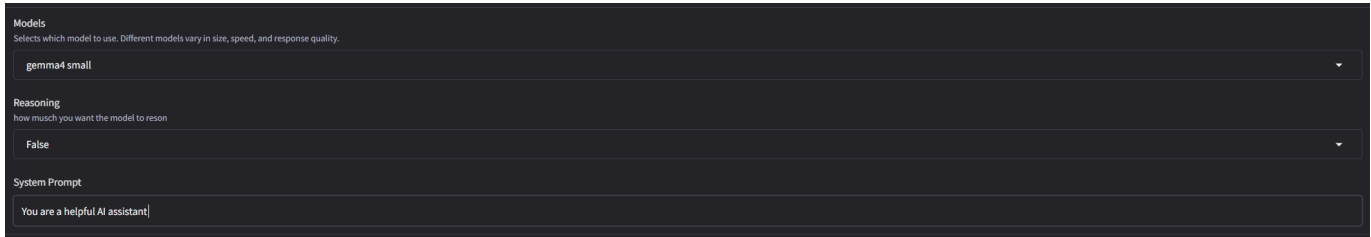


fig 19 LLM model selectie

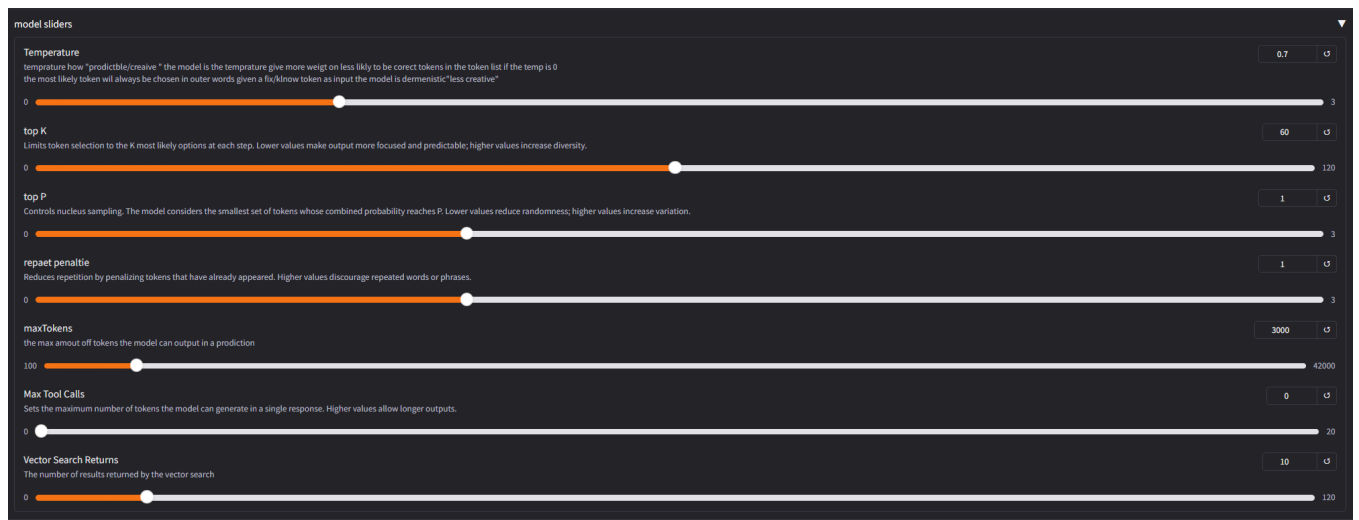


fig 20 slider om de parameters van het model aan te passen

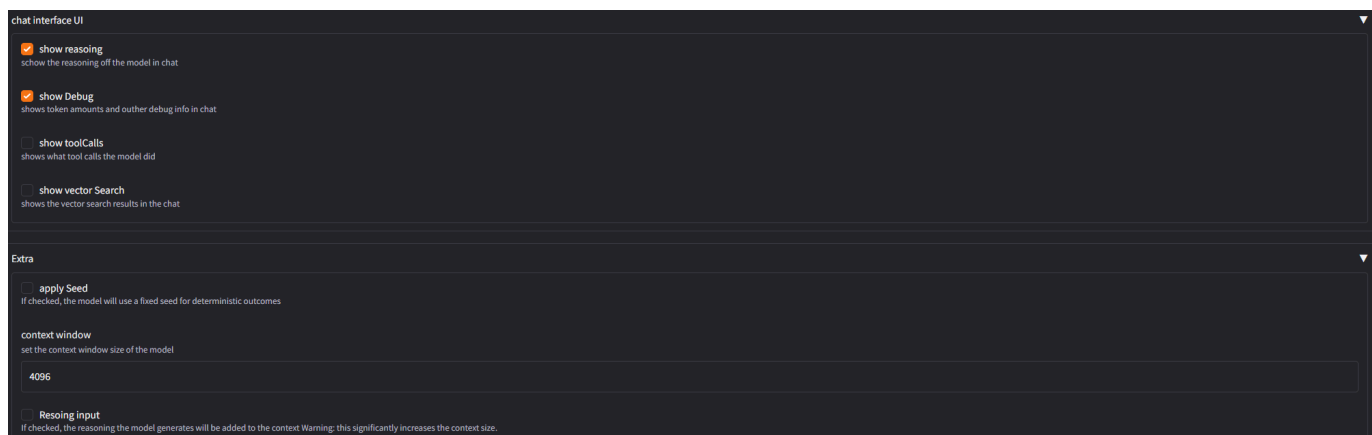


fig 21 extra instellingen en ui optimalisaties

10.1.4 zoeken met technische schema

Op deze pagina kan een PDF of DWG (elektrisch schema) worden geüpload, welke vervolgens wordt toegevoegd aan de KN en omgezet naar een smart PDF. Bestanden kunnen hier ook uit de KN worden verwijderd (fig 22).

Er is een tweede pagina (fig 23) waar een smart PDF uit de KN geselecteerd kan worden. Na selectie kan op onderdelen in de PDF worden geklikt voor een directe doorverwijzing naar de informatiepagina (10.1.2).

je krijgt ook een lijst te met alle gevonden onderdelen op dat schema

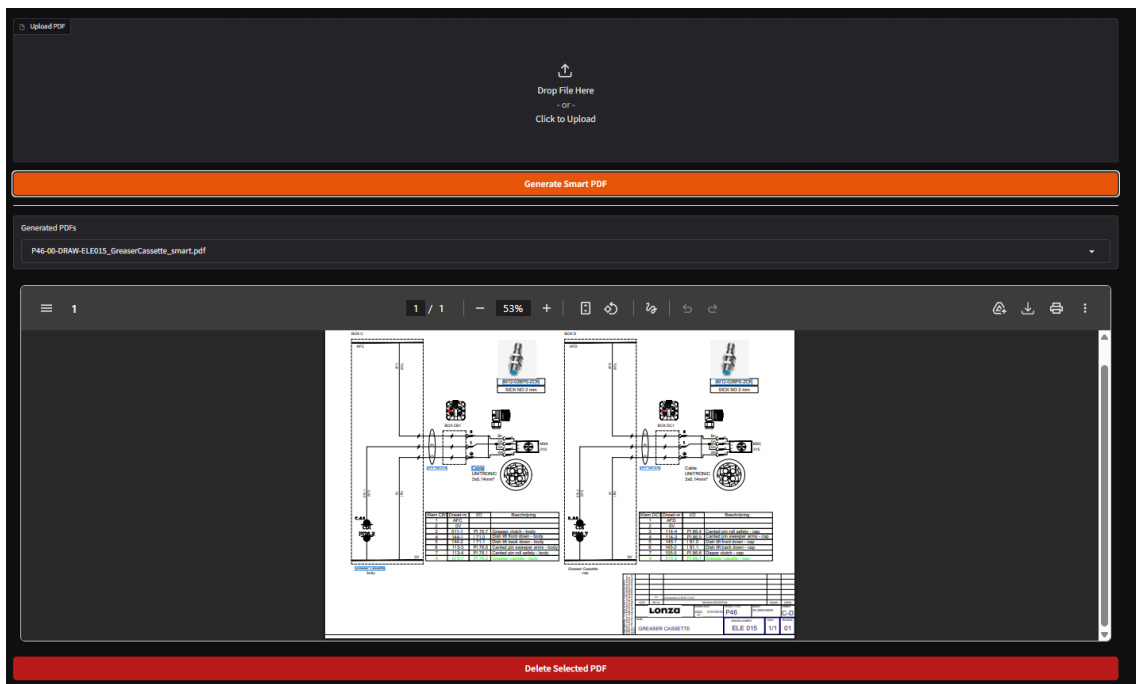


fig 22 pagina om smart pdf te genereren

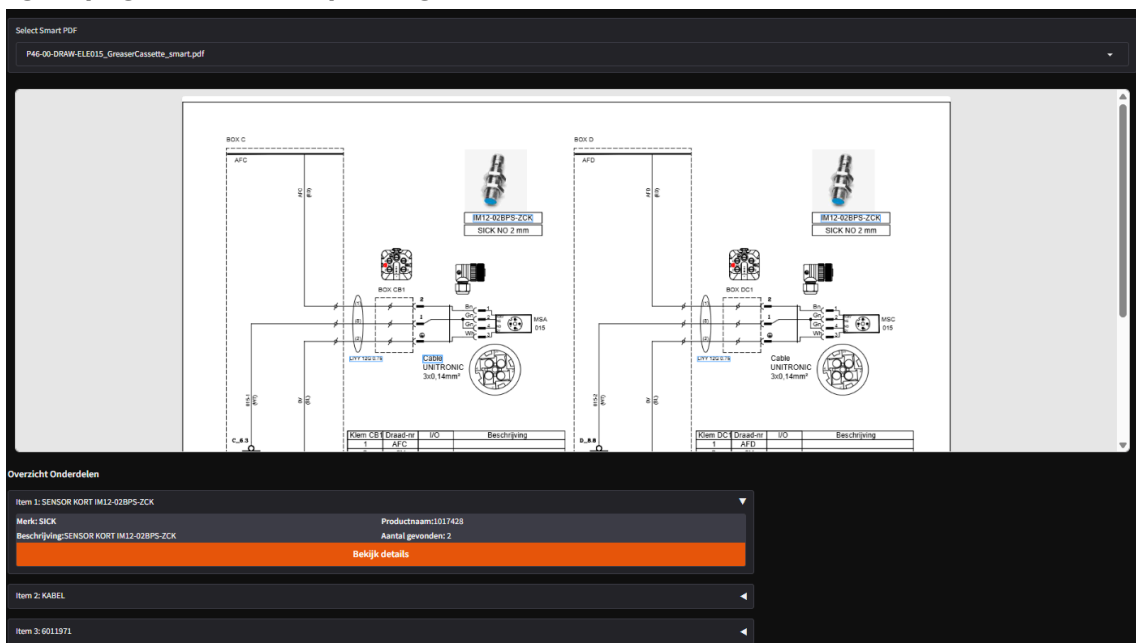


fig 23 pagina om smart pdf te bekijken

10.2 Elektrische schema's, SharePoint en smart PDF's

Bij het genereren van de smart PDF worden niet alleen de onderdelen eruit gehaald, maar ook alle andere info zoals de naam van de tekening, datum, tot welke machine deze behoort, enzovoort. Deze worden opgeslagen als properties in een node wanneer deze PDF in de KN komt. Later, wanneer we bijvoorbeeld alle tekeningen van een bepaalde machine willen toevoegen aan SharePoint, kunnen we deze heel gemakkelijk filteren en uploaden naar SharePoint, of alle tekeningen van een bepaalde datum, enzovoort.

Een idee, dat de stagementor voorstelde, is het plaatsen van QR-codes aan elektrische borden die rechtstreeks naar de correcte smart PDF zouden linken in de applicatie. Hier kan een persoon gewoon naar een echt bord gaan, de QR-code scannen en direct een schema krijgen met een lijst van alle onderdelen en info (10.1.2).

10.3 behaalde doelstellingen en afwijking van het opslag verslag

10.3.1 PLC code

De meeste doelstellingen zijn behaald. Er is een afwijking ten opzichte van het opstart verslag wat betreft de verwerking van PLC-code voor het genereren van beschrijvingen. Onderzoek wees uit dat dit aanzienlijk complexer is dan aanvankelijk ingeschat.

Probleem 1: Er bestaan geen open-source tools voor het openen en uitlezen van PLC-bestanden. Er zijn wel API's beschikbaar waarmee communicatie met de bestanden mogelijk zou zijn maar deze vereisen dat de bestanden eerst in TIA Portal worden geopend. Vervolgens zou een eigen programma de API moeten aansturen, wat het proces complex maakt voor de eindgebruiker.

Probleem 2: dit vormt het grootste probleem. Een PLC-bestand is zeer complex en bestaat uit meerdere onderdelen. Het gaat niet om één enkel programma, maar om een projectstructuur met meerdere programma's, definities voor poorten en bijkomende configuraties. Daarnaast bestaan er verschillende versies en formaten. Oude bestanden kunnen niet altijd met dezelfde software worden geopend. Er is bovendien een verschil tussen STEP 7- en TIA Portal-bestanden, en de structuur en layout verschillen per type. Het correct extraheren van data uit PLC-code zou op zichzelf al een volledig project kunnen zijn

Probleem 3: zelfs indien de data beschikbaar zou zijn, is de meerwaarde beperkt. De enige realistische toepassing zou zijn om de data door te geven aan een LLM om een beschrijving te genereren. Dit is niet echt interessant als AI gericht eindwerk en bovendien moeilijk uitvoerbaar, aangezien het lokaal draaien van een model dat dergelijke complexe en grote data kan verwerken bijna niet haalbaar is op beperkte hardware, een P4000 met 8 GB VRAM.

10.3.2 onderdelen vinden in het magazijn

In plaats van te focussen op de PLC-code, is het mogelijk gemaakt dat personen foto's kunnen nemen van onderdelen om deze terug te vinden in het magazijn. dit stuk dient als vervanging van de plc code

10.4 ongebruikte methodes

Hier worden kort de methoden toegelicht die wel zijn ontwikkeld, maar uiteindelijk niet zijn toegepast.

10.4.1 DWG-extractie

Zoals vermeld in hoofdstuk 1 was de initiële opzet om data direct uit DWG-bestanden te extraheren. Hoewel het systeem hiervoor was ingericht, bleken deze bestanden vaak onvoldoende metadata te bevatten. Daarom is de focus verschoven naar PDF en OCR.

De doelstelling om zowel DWG-files als PDF-files te verwerken, en om PDF-files te genereren vanuit DWG-files is wel behaald. De Smart PDF Generator aanvaardt zowel DWG- als PDF-files als input.

10.4.2 Image recognition

Het eerste plan om onderdelen uit het magazijn te herkennen, was gebruikmaken van image recognition. Dit werd kort besproken in hoofdstuk 5. Hoewel deze methode uiteindelijk niet gebruikt is, was deze wel volledig uitgewerkt. Daarom wordt hier in het kort uitgelegd hoe deze werkt.

Er werd een onderdeel uit het magazijn genomen en hiervan werd een referentievideo gemaakt van ongeveer 40 tot 60 seconden. Daarna werd deze video omgezet naar 3 FPS, voor een totaal van ongeveer 120 tot 180 reference frames. Ook werd de achtergrond van de video verwijderd. Vervolgens werd voor elk frame een embedding gemaakt en opgeslagen in de KN.

Later kon een gebruiker dan een video van 5 tot 10 seconden maken van een onderdeel. Opnieuw werd de achtergrond verwijderd, werden de frames uit de video gehaald en embeddings gemaakt. Daarna werd een voting system gebruikt waarbij voor elke query embedding/frame de dichtstbijzijnde embedding in de KN werd gezocht. Vervolgens kon bekeken worden welk onderdeel de meeste matches had en van daaruit werd dat onderdeel geselecteerd.

Dit werkte heel goed, maar er waren 3 grote nadelen.

1. Het was totaal niet gebruiksvriendelijk voor de eindgebruiker. Deze moest een video van 5 tot 10 seconden maken van een onderdeel, en het proces was ook traag.
2. Elk onderdeel dat teruggevonden moest kunnen worden, moest eerst in de KN geplaatst worden. Dit betekende voor elk onderdeel een video van 30 tot 60 seconden, wat praktisch onhaalbaar was voor 20.000 onderdelen.
3. Er kon moeilijk of geen onderscheid gemaakt worden tussen onderdelen die visueel sterk op elkaar lijken. Bijvoorbeeld twee zekeringen van hetzelfde merk die er identiek uitzien, maar waarvan één 16A is en de andere 20 A. Deze zouden moeilijk van elkaar onderscheiden kunnen worden.

11. conclusie

Dit project toont aan dat het mogelijk is om versnipperde technische bedrijfsinformatie efficiënt te centraliseren met behulp van een Neo4j knowledge graph. Door magazijngegevens, technische schema's en documentatie via expliciete relaties met elkaar te verbinden, werd het opzoeken van componentinformatie sterk vereenvoudigd en geautomatiseerd.

Een belangrijke uitdaging binnen het project was de beperkte kwaliteit van de metadata in de originele DWG-bestanden. Om dit probleem op te lossen, werd een OCR-gebaseerde extractiepipeline ontwikkeld met behulp van PaddleOCR. In combinatie met vector similarity search maakte deze pipeline het mogelijk om componenten uit technische tekeningen automatisch te koppelen aan de juiste magazijninformatie en documentatie.

Tijdens het onderzoek werden verschillende embedding-modellen en zoekmethodieken geëvalueerd. Uit de experimenten bleek dat één lokaal draaiend algemeen embeddingmodel, namelijk *EmbeddingGamma*, de beste balans biedt tussen snelheid, nauwkeurigheid en rekenvereisten. Hoewel methodes zoals Weighted Reciprocal Rank Fusion goede resultaten konden behalen, bleek de extra complexiteit en rekentijd in deze context minder efficiënt. De gekozen oplossing bleef bovendien robuust presteren bij OCR-ruis en imperfecte invoerdata.

Het uiteindelijke systeem resulteerde in een lokaal draaiende applicatie die gebruikers ondersteunt via visuele zoekfunctionaliteit, interactieve technische schema's en een offline RAG-chatbot voor technische ondersteuning. Hierdoor wordt de toegang tot machineonderdelen, documentatie en magazijninformatie sneller en gebruiksvriendelijker gemaakt.

De resultaten tonen aan dat knowledge graphs gecombineerd met OCR, embeddings en lokale AI-technieken een praktische en schaalbare oplossing vormen voor industriële informatieverwerking. Verdere uitbreidingen kunnen zich richten op realtime synchronisatie met het magazijn, verbeterde OCR-nauwkeurigheid en ondersteuning voor bijkomende documenttypes en CAD-formaten en het verwerken van plc code .

Literatuurlijst

[ref1] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34-43. <https://www.scientificamerican.com/article/the-semantic-web/>

[ref 2] Ileri, M., et al. (2020). Hypergraph representations of knowledge graphs. [arXiv preprint]. <https://arxiv.org/abs/2603.03304>

[ref 3] Neo4j Labs. (2024). *Neo4j LLM Knowledge Graph Builder* [Documentatie en limieten van GenAI integraties]. Neo4j. <https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/>

[ref4] neo 4j vector vector simulay search
<https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/>

[ref 8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela
(2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
arXiv:2005.11401. Link: (<https://arxiv.org/abs/2005.11401>)

[ref 9] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs.
link:<https://arxiv.org/abs/1603.09320>

[ref 10] Nogueira, R., & Lin, J. (2019). From doc2query to docTTTTTquery.
arXiv:1904.08375. Link: <https://arxiv.org/abs/1904.08375>

[ref 12] Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models.
arXiv:2210.03629. Link: <https://arxiv.org/abs/2210.03629>

[ref 13] Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761. Link: <https://arxiv.org/abs/2302.04761>

[ref 14] Lund, W. B., Kennard, D. J., & Ringger, E. K. (2011). Combining multiple OCR systems to improve text recognition. Document Recognition and Retrieval XVIII. Link: <https://dl.acm.org/doi/10.1117/12.876523>

[ref 15] Unel, F. O., Ozkalayci, B. O., & Cigla, C. (2019). The Power of Tiling for Small Object Detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops. Link:
https://openaccess.thecvf.com/content_CVPRW_2019/html/UAV_Vision/Unel_The_Power_of_Tiling_for_Small_Object_Detection_CVPRW_2019_paper.html

[ref 16] G. V. Cormack, C. L. A. Clarke, and S. Büttcher (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. SIGIR 2009. Link: (<https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>)

[ref 17] Jasper Snoek, Hugo Larochelle, Ryan P. Adams (2012). Practical Bayesian Optimization of Machine Learning Algorithms. Advances in Neural Information Processing Systems 25 (NIPS 2012), arXiv:1206.2944. Link: (<https://arxiv.org/abs/1206.2944>)

[ref 18] Docling Project. (2024). Docling: Document processing for AI workflows. GitHub repository. Link: <https://github.com/docling-project/docling>

afbeeldingen

Dit zijn de bronnen van afbeeldingen/schema's die ik online gehaald heb en niet zelf heb gemaakt.

1.simple rage pile line schema link:

<https://pub.towardsai.net/what-is-rag-a-clear-and-simple-explanation-a3980bd3a61a>



CONTACT

Robin herickx | student
r0983377@student.thomasmore.be

VOLG ONS

www.thomasmore.be
fb.com/ThomasMoreBE
#WeAreMore

THOMAS
MORE